

UNIVERSIDADE FEDERAL DO CEARÁ
DEPARTAMENTO DE ENGENHARIA DE COMPUTAÇÃO
LABORATÓRIO DE ENGENHARIA DE SISTEMAS DE
COMPUTAÇÃO (LESC)

**Documentação de Arquitetura de Hardware:
Multiplexador Reconfigurável de Códigos de
Correção de Erros (ECC) em Tempo Real**

Autor:

Matheus Parente Reis

Fortaleza, Ceará
21 de julho de 2026

Sumário

1	Introdução	4
2	Fundamentação Teórica	5
2.1	Linguagens de Descrição de Hardware (HDL)	5
2.2	SystemVerilog	6
2.3	Constructos Críticos em SystemVerilog	6
2.3.1	Sinais, Tipos de Dados e Atribuições	6
2.3.2	Módulos, Direcionalidade e Hierarquia de Design (Top-Level)	9
2.3.3	Vetores (Barramentos) e Indexação	11
2.3.4	Representação de Valores e Estados Lógicos	13
2.3.5	Lógica Combinacional (<code>always_comb</code>)	15
2.3.6	Operadores em Nível RTL	17
2.3.7	Portas Bidirecionais (<code>inout</code>) e Buffers <i>Tri-state</i>	18
3	Códigos de Correção de Erros (ECC)	21
3.1	Conceitos Gerais de Integridade de Dados	21
3.1.1	Introdução à Confiabilidade em Memória	21
3.1.2	Fundamentação Teórica: Do Bit de Paridade ao Código de Hamming	21
3.1.3	Fluxo Operacional de Codificação e Decodificação	23
3.2	Matrix Region Selection Code (MRSC)	24
3.2.1	Definição Teórica e Formulação Matemática	24
3.2.2	Exemplo de Codificação e Decodificação	26
3.2.3	Análise de Vantagens e Desvantagens	30
3.3	Triple Burst Error Correction Based on Region Selection Code (TBEC-RSC)	30
3.3.1	Definição Teórica e Formulação Matemática	30

4	Requisitos do Projeto e Ambiente de Teste	32
4.1	Especificações de Hardware e Placa Alvo	32
4.2	Cenários de Validação e Requisitos de Teste	33
5	Arquitetura do Hardware e Implementação (Abordagem Bottom-Up)	34
5.1	Nível 1: Codecs Individuais	35
5.1.1	Codificador e Decodificador MRSC	35
5.1.2	Codificador e Decodificador TBEC-RSC	41
5.2	Nível 2: Multiplexador de ECC (<i>ecc_design</i>)	48
5.3	Nível 3: Módulo Top-Level (<i>fpga_top_design</i>)	52
5.4	Fluxos Críticos de Sincronismo	57
5.4.1	Fluxo de Operação de Escrita (<i>Write Flow</i>)	57
5.4.2	Fluxo de Operação de Leitura (<i>Read Flow</i>)	57
5.5	Diagrama de Blocos Estrutural	58
6	Controle Externo e Interface com Periféricos	58
6.1	Integração com Microcontrolador (STM32)	58
6.2	Interface com Memória Externa (SRAM)	59
7	Resultados e Considerações Finais	60
8	Tutorial: Gravação do Bitstream na FPGA com o Libero SoC 2025.2	61
8.1	Abertura do Projeto no Libero SoC	61
8.2	Síntese do Design	62
8.3	Place and Route	63
8.4	Geração do Bitstream	64
8.5	Seleção do Programador (FlashPro) e Gravação	64
8.6	Resumo do Fluxo	66

8.7	Criação de um Novo Projeto no Libero SoC	66
8.7.1	Página de Boas-Vindas	67
8.7.2	Detalhes do Projeto (<i>Project Details</i>)	67
8.7.3	Seleção do Dispositivo (<i>Device Selection</i>)	67
8.7.4	Configurações do Dispositivo (<i>Device Settings</i>)	68
8.7.5	Modelo de Design (<i>Design Template</i>)	69
8.7.6	Importação dos Arquivos-Fonte (<i>Add HDL Sources</i>)	70
8.7.7	Definição do Módulo de Topo (<i>Set As Root</i>)	71
8.7.8	Síntese Inicial e Verificação das Constraints	72
8.7.9	Abertura do I/O Editor	73
8.7.10	Atribuição dos Pinos no I/O Editor	74
8.7.11	Continuação do Fluxo a partir do Place and Route	75

1 Introdução

Em ambientes extremos, como missões aeroespaciais e operações de satélites, os sistemas eletrônicos enfrentam severas restrições de energia e estão constantemente expostos a níveis elevados de radiação ionizante. Essa radiação interage com os semicondutores e provoca perturbações lógicas, destacando-se o SEU (Single Event Upset - alteração de um único bit) e o MCU (Multiple Cell Upset - alteração de múltiplos bits adjacentes). Quando essas falhas ocorrem em regiões críticas de armazenamento, como em módulos de memória SRAM (Static Random Access Memory), elas podem corromper dados vitais e desencadear erros irreversíveis na operação do sistema.

Para mitigar esses efeitos e garantir a confiabilidade dos dados, empregam-se os algoritmos de ECC (Error Correction Code). Esses códigos inserem redundância matemática à informação, permitindo que o hardware detecte e corrija os bits invertidos pela radiação. Contudo, existe um compromisso de projeto (trade-off) inerente a essa técnica: quanto mais robusto for o ECC — ou seja, quanto maior a sua capacidade de corrigir múltiplos erros — maior será o custo computacional e o consumo energético associado à sua execução.

Diante da escassez de energia nesses ambientes adversos, este projeto propõe uma arquitetura de hardware adaptativa. O sistema consiste em um módulo lógico, implementado em uma FPGA, que atua como um multiplexador dinâmico de códigos de correção. Por meio de sinais de controle externos, o circuito seleciona em tempo real qual algoritmo de ECC será aplicado durante as operações de leitura e escrita em duas memórias SRAM. Essa abordagem permite que o sistema alterne dinamicamente entre diferentes níveis de proteção, escolhendo o código mais adequado para a situação específica e otimizando o balanço entre a robustez contra falhas e a eficiência energética do satélite.

2 Fundamentação Teórica

2.1 Linguagens de Descrição de Hardware (HDL)

Para compreender o papel das Linguagens de Descrição de Hardware (HDL), é fundamental diferenciá-las das linguagens de programação de software convencionais, como C, C++ ou Python.

Uma linguagem de programação de software atua sobre um hardware que já existe fisicamente — como um microcontrolador, um processador ou um computador. O código escrito pelo programador fornece uma sequência de instruções temporais que esse hardware pré-implementado deve executar passo a passo para atingir um resultado.

Em contrapartida, uma linguagem HDL não programa um componente existente; ela descreve a própria estrutura física e o comportamento do circuito que será fabricado ou sintetizado. Ao escrever em HDL, o projetista não está enviando comandos para um processador, mas sim determinando como portas lógicas, flip-flops, multiplexadores e interconexões elétricas devem ser organizados no nível do silício (ou nas células de uma FPGA).

Ou seja, enquanto a linguagem de programação diz ao hardware o que fazer de forma sequencial, o HDL define o que o hardware é. Isso confere ao HDL uma natureza inerentemente espacial e concorrente, onde múltiplos blocos descritos no código funcionam literalmente ao mesmo tempo, de forma paralela.

2.2 SystemVerilog

O SystemVerilog é uma Linguagem de Descrição e Verificação de Hardware (HDVL) que estende o Verilog clássico, adicionando recursos modernos e maior abstração. Sua arquitetura é baseada em módulos interconectados que processam entradas e geram saídas. A linguagem atua em duas frentes principais:

- **Modelagem de Hardware:** Embora suporte abstrações mais altas (comportamental) e mais baixas (nível de portas), este projeto focará estritamente no **Nível RTL** (*Register Transfer Level*). O RTL descreve de forma clara o fluxo de dados entre registradores e é o nível de abstração padrão exigido para que o código seja fisicamente sintetizado em FPGAs ou ASICs. A modelagem utiliza blocos explícitos: `always_comb` para lógica combinacional, `always_ff` para lógica sequencial, e `always_latch` para *latches*.
- **Verificação Funcional:** Introduz ferramentas avançadas (orientação a objetos, randomização, asserções) na criação de *testbenches*. Estes são ambientes de simulação não sintetizáveis desenvolvidos exclusivamente para atestar a validade lógica do hardware antes da sua implementação física.

2.3 Constructos Críticos em SystemVerilog

2.3.1 Sinais, Tipos de Dados e Atribuições

Para compreender a modelagem estrutural no SystemVerilog, é fundamental analisar os tipos físicos e lógicos de sinais (`wire`, `reg` e `logic`), bem como os métodos de atribuição que definem o comportamento do circuito gerado no silício.

O `wire` é uma herança do Verilog clássico que representa literalmente um barramento de conexão entre diferentes partes de um circuito digital. Ele não armazena valor (não possui memória) e apenas transmite um sinal elétrico. Por não reter estados lógicos, um `wire` deve ser continuamente "direcionado" (*driven*) por uma fonte, como a saída de uma porta lógica ou uma atribuição contínua.

O `reg` é exigido pelo Verilog clássico sempre que é necessário atribuir um valor dentro de um bloco procedural (como um `always`). Apesar do nome, um `reg` é apenas uma

variável que retém seu valor até a próxima atribuição. Ele só será sintetizado fisicamente como um *flip-flop* (registrador) se o bloco procedural que o controla for sensível a um sinal de *clock*.

O SystemVerilog solucionou ambiguidades e erros de tipagem introduzindo o tipo `logic`. Ele é um tipo de dado versátil que substitui tanto o `wire` quanto o `reg`. A ferramenta de síntese infere automaticamente se a variável `logic` será um fio combinacional ou um elemento de memória, com base exclusivamente na forma como o código foi descrito. A restrição fundamental do `logic` é não permitir múltiplos *drivers* simultâneos, o que previne curtos-circuitos lógicos no projeto.

Além dos tipos de sinais, a correta descrição do hardware depende do operador de atribuição utilizado:

- **Atribuição Contínua (`assign`):** Utilizada fora de blocos procedurais para direcionar sinais de forma contínua e permanente. É estritamente combinacional.
- **Atribuição Bloqueante (`=`):** Utilizada dentro de blocos procedurais (como `always_comb`). A execução ocorre sequencialmente (uma linha é avaliada antes da próxima). É o padrão correto para modelar **lógica combinacional** complexa.
- **Atribuição Não-Bloqueante (`<=`):** Utilizada dentro de blocos procedurais sensíveis a bordas de sinal (como `always_ff`). Todas as operações daquele bloco são avaliadas simultaneamente ao final do ciclo de *clock*. É o padrão absoluto para modelar **lógica sequencial** (registradores e memórias), pois reflete a concorrência real do hardware.

Abaixo, os exemplos de código demonstram a aplicação prática do tipo `logic` associado aos diferentes métodos de atribuição:

```
1 module logic_combinacional (  
2     input  logic a,  
3     input  logic b,  
4     output logic c, // Atuando como fio direto  
5     output logic d  // Atuando como variavel procedimental  
6 );  
7
```

```

8      // 1. Atribuicao continua (fora de blocos)
9      // A ferramenta infere um caminho permanente
10     assign c = a ^ b;
11
12     // 2. Atribuicao bloqueante (dentro de always_comb)
13     // A avaliacao ocorre em sequencia imediata
14     always_comb begin
15         d = a & b;
16     end
17
18 endmodule

```

Listing 1: Uso de logic com atribuição contínua e bloqueante (Lógica Combinacional).

```

1 module logic_sequencial (
2     input  logic clk,
3     input  logic reset,
4     input  logic d_in,
5     output logic q_out // Atuando como memoria/flip-flop
6 );
7
8     // Bloco procedural sincrono
9     // A ferramenta infere um flip-flop tipo D
10    always_ff @(posedge clk) begin
11        if (reset)
12            q_out <= 1'b0; // Atribuicao nao-bloqueante
13        else
14            q_out <= d_in; // Ocorre simultaneamente na borda de
15                          subida
16    end
17 endmodule

```

Listing 2: Uso de logic com atribuição não-bloqueante (Lógica Sequencial).

2.3.2 Módulos, Direcionalidade e Hierarquia de Design (Top-Level)

No SystemVerilog, a unidade fundamental de construção de hardware é o **módulo** (`module`). Ele funciona como um bloco construtivo isolado (uma "caixa preta") que contém uma lógica interna e se comunica com o mundo exterior estritamente através de suas portas de interface. Essa abordagem modular permite que sistemas complexos sejam divididos em partes menores, testáveis e reutilizáveis.

A comunicação de um módulo com os demais componentes do circuito é rigidamente controlada pela declaração de direcionalidade de seus pinos:

- **input:** Define um sinal de entrada. Permite que o módulo receba dados operacionais externos. No escopo interno do módulo, esse sinal é tratado estritamente como leitura, não podendo ser alterado ou reatribuído pela lógica interna.
- **output:** Define o caminho de saída, enviando as respostas processadas para outros blocos. Sinais declarados como saídas podem ser lidos e ativamente modificados pelo algoritmo estrutural interno do módulo.

A verdadeira força do SystemVerilog está na sua capacidade hierárquica. Circuitos complexos, como um multiplexador de ECC, não são escritos em um único bloco de código gigante. Em vez disso, módulos de baixo nível (como decodificadores individuais) são descritos separadamente e, em seguida, **instanciados** dentro de um módulo de nível superior, conhecido como **Top-Level** ou **Top Design**.

No módulo Top, o projetista declara sinais lógicos internos (`logic`) que atuarão como fios de ligação para conectar a porta `output` de um submódulo à porta `input` de outro, estabelecendo o fluxo de dados (*datapath*) completo da arquitetura.

Abaixo, um exemplo prático demonstrando a criação de um submódulo e sua posterior instanciação em um Top Design, evidenciando o mapeamento das portas:

```

1 // -----
2 // 1. Submodulo de baixo nivel (Ex: Logica Combinacional)
3 // -----
4 module bloco_and_simples (
5     input  logic a, // Porta de entrada (somente leitura interna)
6     input  logic b, // Porta de entrada
7     output logic y  // Porta de saida (modificavel internamente)
8 );
9
10 // A logica interna direciona o resultado para a saida
11 assign y = a & b;
12
13 endmodule
14
15 // -----
16 // 2. Modulo Top-Level (Hierarquia Superior)
17 // -----
18 module top_design_hierarquico (
19     input  logic clk,
20     input  logic sinal_ext_1,
21     input  logic sinal_ext_2,
22     output logic resultado_final
23 );
24
25 // Fio logico interno usado para interconectar os blocos
26 logic fio_intermediario;
27
28 // Instanciacao do submodulo "bloco_and_simples"
29 // A sintaxe .porta_interna(sinal_externo) mapeia as conexoes
30 bloco_and_simples inst_and_01 (
31     .a (sinal_ext_1),          // Conecta entrada externa ao pino
32     'a',
33     .b (sinal_ext_2),          // Conecta entrada externa ao pino
34     'b',

```

```

33         .y (fio_intermediario) // Conecta a saída 'y' ao fio
34         interno
35     );
36
37     // O Top Design também pode ter sua própria lógica
38     // Aqui, registramos o resultado intermediário na borda do
39     clock
40     always_ff @(posedge clk) begin
41         resultado_final <= fio_intermediario;
42     end
43 endmodule

```

Listing 3: Exemplo de criação de submódulo e instanciação em um Top Design.

2.3.3 Vetores (Barramentos) e Indexação

Em circuitos digitais, é raro trafegar a informação utilizando apenas fios individuais (1 bit). Para transmitir pacotes de dados, endereços de memória e instruções complexas, utiliza-se o conceito de barramento: um agrupamento paralelo de fios. No SystemVerilog, esses barramentos são modelados através de **vetores** (*packed arrays*).

A declaração padrão de um vetor em SystemVerilog segue a sintaxe **tipo [MSB:LSB] nome_do_sinal**, onde:

- **MSB (*Most Significant Bit*)**: O índice do bit mais significativo (o de maior peso, posicionado à esquerda).
- **LSB (*Least Significant Bit*)**: O índice do bit menos significativo (o de menor peso, posicionado à direita).

A convenção mais adotada e segura na indústria é o formato **[N-1 : 0]**, onde N é o tamanho total do barramento. Por exemplo, um barramento de 8 bits é declarado como **[7:0]**.

A linguagem permite não apenas operar o barramento como um bloco único, mas

também manipular seus elementos individualmente através de indexação direta ou fatiamento de conjuntos menores (*part-select*):

- **Indexação de Bit Único:** Acessa um fio isolado dentro do barramento (Ex: `barramento[2]`).
- **Fatiamento Contínuo:** Extrai um subconjunto de fios sequenciais. A sintaxe utiliza o formato `[MSB_desejado : LSB_desejado]` (Ex: `barramento[5:2]`).

Abaixo, apresenta-se um exemplo prático das técnicas de manipulação vetorial em nível RTL:

```
1 module manipulacao_vetores (  
2     input  logic [15:0] endereco_completo, // Barramento de 16 bits  
        (15 ate 0)  
3     output logic [7:0]  endereco_alto,      // Barramento de 8 bits  
4     output logic [7:0]  endereco_baixo,    // Barramento de 8 bits  
5     output logic        bit_paridade,      // Fio individual de 1  
        bit  
6     output logic [3:0]  nibble_central     // Barramento de 4 bits  
7 );  
8  
9     always_comb begin  
10         // 1. Fatiamento (Slicing) Superior e Inferior  
11         // Dividindo o endereco de 16 bits em dois pacotes de 8  
        bits  
12         endereco_alto  = endereco_completo[15:8];  
13         endereco_baixo = endereco_completo[7:0];  
14  
15         // 2. Acesso a um Bit Isolado  
16         // Capturando apenas o bit mais significativo do endereco  
17         bit_paridade   = endereco_completo[15];  
18  
19         // 3. Fatiamento Central  
20         // Extraindo 4 bits do meio do barramento (do bit 9 ao bit  
        6)
```

```

21 nibble_central = endereco_completo[9:6];
22 end
23
24 endmodule

```

Listing 4: Exemplo de declaracao de vetores, indexacao e fatiamento.

2.3.4 Representação de Valores e Estados Lógicos

No SystemVerilog, a manipulação de dados em nível de hardware exige uma sintaxe rigorosa que define não apenas o valor numérico, mas também a largura exata em bits e a base matemática utilizada. Além disso, diferente da programação de software clássica que lida com lógica puramente booleana (verdadeiro ou falso), a modelagem de circuitos digitais opera com um sistema de quatro estados lógicos fundamentais:

- **0:** Nível lógico baixo (Falso / GND / 0 Volts).
- **1:** Nível lógico alto (Verdadeiro / VCC).
- **X (ou x):** Estado desconhecido (*Unknown*). Representa um valor lógico imprevisível. Ocorre geralmente quando um registrador não foi inicializado (resetado) antes do uso ou quando há um curto-circuito (múltiplas fontes tentando escrever '0' e '1' no mesmo fio simultaneamente).
- **Z (ou z):** Alta impedância (*High Impedance*). Indica que o terminal está eletricamente flutuando (desconectado). É um estado vital para o controle de barramentos *tri-state* compartilhados, que será visto posteriormente.

Para atribuir valores constantes a vetores ou barramentos, a linguagem utiliza a seguinte notação padronizada: `[tamanho]'[base][valor]`.

- **Tamanho:** Indica a quantidade exata de bits que o número ocupará no hardware. Se omitido, o sintetizador assumirá o tamanho padrão do sistema (geralmente 32 bits), o que pode gerar avisos de compilação por truncamento.

- **Base:** Representada por uma letra precedida de um apóstrofo. As opções mais comuns são 'b (Binário), 'h (Hexadecimal) e 'd (Decimal). Também há suporte para Octal ('o).
- **Valor:** A sequência numérica na base especificada. Para facilitar a leitura de números muito extensos no código, o SystemVerilog permite o uso livre de *underline* (_) para separar grupos de dígitos.

Abaixo, um exemplo demonstrando a correta atribuição desses valores e estados lógicos em diferentes bases:

```

1 module exemplos_de_dados (
2     output logic [7:0] barramento_bin,
3     output logic [7:0] barramento_hex,
4     output logic [7:0] barramento_dec,
5     output logic [3:0] estados_logicos
6 );
7
8 always_comb begin
9     // 1. Representacao Binaria ('b)
10    // 8 bits de tamanho, base binaria, valor 10101010
11    // 0 underline e ignorado pelo hardware, servindo apenas
12    para visualizacao
13    barramento_bin = 8'b1010_1010;
14
15    // 2. Representacao Hexadecimal ('h)
16    // 8 bits de tamanho, base hex, valor FF (equivale a 255 em
17    decimal)
18    // Muito util para simplificar vetores extensos (enderecos
19    de memoria)
20    barramento_hex = 8'hFF;
21
22    // 3. Representacao Decimal ('d)
23    // 8 bits de tamanho, base decimal, valor 100
24    barramento_dec = 8'd100;

```

```

23      // 4. Insercao de estados Z e X
24      // Um vetor de 4 bits recebendo a combinacao:
25      // 1, 0, Z (Alta Impedancia), X (Desconhecido)
26      estados_logicos = 4'b10ZX;
27  end
28
29 endmodule

```

Listing 5: Exemplos de atribuicao de valores, bases numericas e estados logicos.

2.3.5 Lógica Combinacional (`always_comb`)

A lógica combinacional refere-se a circuitos cujas saídas dependem única e exclusivamente do estado atual de suas entradas, sem qualquer retenção de estado, *clock* ou memória. Em SystemVerilog, a modelagem procedural desse tipo de circuito é realizada através do bloco explícito `always_comb`.

Historicamente, o Verilog clássico utilizava o bloco genérico `always @(*)` para inferir esse comportamento. Contudo, a introdução do `always_comb` trouxe melhorias rigorosas para o fluxo de design:

- **Intenção Clara de Síntese:** Informa explicitamente à ferramenta que aquele bloco deve ser transformado apenas em portas lógicas puras (AND, OR, multiplexadores, etc.).
- **Prevenção de *Latches* Inadvertidos:** Em um circuito combinacional real, a saída deve possuir um valor definido para todas as combinações possíveis de entrada. Se o projetista esquecer de mapear uma condição (por exemplo, escrever um `if` e esquecer o `else`), a ferramenta de síntese tentará manter o valor anterior da variável, inferindo uma memória acidental chamada *latch*. O bloco `always_comb` força as ferramentas de compilação a emitirem alertas críticos caso um *latch* seja detectado, protegendo o determinismo do hardware.
- **Atribuição Bloqueante:** Dentro deste bloco, utiliza-se estritamente o operador de atribuição bloqueante (`=`), garantindo que as equações sejam resolvidas em cascata e de forma imediata.

Abaixo, apresenta-se um exemplo prático de um Multiplexador 4:1, uma estrutura clássica puramente combinacional. O uso da estrutura condicional `case` ilustra como o bloco avalia os sinais de controle em tempo real.

```
1 module mux_4to1 (  
2     input  logic [3:0] entradas, // Vetor de 4 bits contendo as  
   entradas  
3     input  logic [1:0] sel,      // Sinal de selecao de 2 bits (00,  
   01, 10, 11)  
4     output logic          saida  // Fio de saida combinacional  
5 );  
6  
7     // Bloco procedural garantido como combinacional  
8     always_comb begin  
9         // A avaliacao ocorre sempre que 'entradas' ou 'sel'  
   mudarem  
10        case (sel)  
11            2'b00: saida = entradas[0]; // Roteia o bit 0  
12            2'b01: saida = entradas[1]; // Roteia o bit 1  
13            2'b10: saida = entradas[2]; // Roteia o bit 2  
14            2'b11: saida = entradas[3]; // Roteia o bit 3  
15  
16            // O caso 'default' e uma boa pratica critica em  
   always_comb  
17            // Ele garante que a 'saida' sempre receba um valor,  
   mesmo  
18            // se 'sel' assumir estados desconhecidos (como 'X' ou  
   'Z'),  
19            // prevenindo a inferencia acidental de latches.  
20            default: saida = 1'b0;  
21        endcase  
22    end  
23  
24 endmodule
```

Listing 6: Implementacao de um Multiplexador 4:1 utilizando `always_comb`.

2.3.6 Operadores em Nível RTL

A modelagem RTL (Register Transfer Level) no SystemVerilog depende de um conjunto robusto de operadores para descrever a manipulação de dados em hardware. A escolha correta do operador dita diretamente como as portas lógicas e os circuitos aritméticos serão inferidos pelo sintetizador.

Os principais operadores dividem-se nas seguintes categorias:

- **Aritméticos** (+, -, *, /): Realizam operações matemáticas. É importante notar que, em síntese física de hardware, operações como soma e subtração inferem somadores lógicos, enquanto multiplicações e divisões consomem uma quantidade massiva de área de silício e devem ser usadas com cautela.
- **Lógicos** (&, ||, !): Avaliam expressões de forma global, retornando um único bit de verdade (1 ou 0). São empregados quase exclusivamente em tomadas de decisão condicionais (ex: dentro de instruções if). **Bit a Bit / Bitwise** (&, |, ^, ~): Operam individualmente e paralelamente em cada bit de um vetor. O operador XOR (^) é a base matemática fundamental para a construção de qualquer gerador de paridade ou código corretor de erros (ECC).
- **Deslocamento / Shift** («, »): Movem os bits de um vetor para a esquerda ou direita. Em nível de hardware, um deslocamento por um valor constante geralmente não consome portas lógicas, consistindo apenas em uma religação física cruzada dos fios.
- **Relacionais** (==, !=, >, <): Inferem circuitos comparadores de magnitude ou igualdade entre dois barramentos.
- **Concatenação** {}: Permite agrupar múltiplos sinais independentes ou pedaços de vetores para formar um barramento maior. É um dos operadores estruturais mais poderosos do SystemVerilog.
- **Condiciona Ternário** (? :): A forma mais limpa e direta de inferir um multiplexador na linguagem. Avalia uma condição e roteia um dos dois caminhos possíveis.

Abaixo, um exemplo prático aplicando operadores essenciais no contexto de estruturação de dados em RTL:

```

1 module operadores_rtl_exemplo (
2     input  logic [3:0] dado_a,
3     input  logic [3:0] dado_b,
4     input  logic      selecao,
5     output logic [7:0] barramento_unido,
6     output logic [3:0] paridade,
7     output logic [3:0] saida_mux
8 );
9
10 always_comb begin
11     // 1. Operador Bit a Bit (XOR)
12     // Aplica o XOR bit a bit entre os dois vetores
13     paridade = dado_a ^ dado_b;
14
15     // 2. Operador de Concatenacao
16     // Agrupa os dois vetores de 4 bits em um unico barramento
17     // de 8 bits
18     barramento_unido = {dado_a, dado_b};
19
20     // 3. Operador Condicional Ternario
21     // Infere um multiplexador 2:1 de forma direta
22     // Se 'selecao' for 1, roteia 'dado_a'. Senao, roteia '
23     // dado_b'.
24     saida_mux = selecao ? dado_a : dado_b;
25 end
26 endmodule

```

Listing 7: Exemplos de operadores bitwise, concatenacao e ternario em RTL.

2.3.7 Portas Bidirecionais (inout) e Buffers *Tri-state*

Em arquiteturas de hardware complexas, a quantidade de pinos físicos de um chip é um recurso extremamente escasso e custoso. Para otimizar essa limitação, utilizam-se

portas bidirecionais (declaradas como `inout` em SystemVerilog), que permitem que um mesmo pino elétrico atue tanto como entrada quanto como saída de dados em momentos distintos.

No entanto, conectar múltiplos módulos em um mesmo fio físico cria um risco crítico: se dois componentes tentarem enviar sinais simultaneamente (por exemplo, um enviando '1' e o outro '0'), ocorrerá um curto-circuito lógico que corromperá os dados e poderá danificar fisicamente o circuito. A solução universal para o controle de barramentos compartilhados é o **Buffer *Tri-state*** (Buffer de Três Estados).

Um buffer *tri-state* atua como um interruptor eletrônico que adiciona um terceiro estado lógico ao circuito, operando das seguintes formas:

1. **Nível Lógico 0 (Baixo):** O driver envia 0 Volts para o barramento.
2. **Nível Lógico 1 (Alto):** O driver envia a tensão de alimentação (ex: 3.3V) para o barramento.
3. **Estado 'Z' (Alta Impedância):** O driver é desligado eletricamente do barramento. Ele não envia '0' nem '1', funcionando como um circuito aberto (fio desconectado).

A aplicação mais didática do buffer *tri-state* ocorre na comunicação entre um microcontrolador (mestre) e uma memória estática (SRAM) externa. Ambos compartilham um único barramento de dados. O fluxo opera sob controle rigoroso para evitar colisões:

- **Ciclo de Escrita (*Write*):** O microcontrolador deseja gravar uma informação. Ele habilita seu próprio buffer *tri-state*, injetando os dados ('0's e '1's) no barramento. Simultaneamente, a memória RAM coloca seu pino bidirecional em estado de **alta impedância ('Z')** e atua apenas como "ouvinte", lendo os dados que estão trafegando e salvando-os.
- **Ciclo de Leitura (*Read*):** O microcontrolador solicita uma informação. Ele imediatamente coloca o seu próprio buffer em estado de **alta impedância ('Z')** e para de enviar sinais. Percebendo a requisição, a memória RAM habilita seu buffer e "toma posse" do barramento, conduzindo os dados armazenados de volta ao microcontrolador.

Abaixo, a implementação típica de um pino `inout` em SystemVerilog utilizando o operador ternário para modelar o controle do buffer *tri-state*:

```
1 module controlador_sram (  
2     input  logic      clk,  
3     input  logic      write_enable, // Sinal de controle (1 =  
4     Escrita, 0 = Leitura)  
5     input  logic [7:0] dado_interno, // Dado que desejamos enviar  
6     para a RAM  
7     output logic [7:0] dado_recebido, // Dado que lemos da RAM  
8     inout  logic [7:0] barramento_io // Pino fisico bidirecional  
9     conectado a RAM  
10 );  
11  
12 // -----  
13 // Logica de Escrita (Driver do Buffer Tri-state)  
14 // -----  
15 // Se write_enable for 1, o barramento recebe o 'dado_interno'.  
16 // Se write_enable for 0, o barramento assume '8'bZ',  
17 desligando  
18 // o driver deste modulo e permitindo que a RAM controle o fio.  
19 assign barramento_io = (write_enable) ? dado_interno : 8'  
20 bZZZZZZZZ;  
21  
22 // -----  
23 // Logica de Leitura  
24 // -----  
25 // O sinal do pino inout pode ser lido continuamente,  
26 // independentemente do estado do buffer.  
27 always_comb begin  
28     dado_recebido = barramento_io;  
29 end  
30 endmodule
```

Listing 8: Modelagem de um Buffer Tri-state em SystemVerilog.

3 Códigos de Correção de Erros (ECC)

3.1 Conceitos Gerais de Integridade de Dados

3.1.1 Introdução à Confiabilidade em Memória

Em ambientes computacionais modernos, a integridade da informação é um desafio constante. Dispositivos de memória, como as SRAMs, são suscetíveis a perturbações eletromagnéticas e radiação ionizante, fenômenos que podem causar a inversão indesejada de níveis lógicos (conhecidos como *soft errors* ou bit-flips). Em sistemas críticos — como aplicações aeroespaciais, automotivas e servidores de alta disponibilidade — a ocorrência de tais falhas pode comprometer o funcionamento total do sistema. Para mitigar esse risco, utiliza-se o ECC (*Error-Correcting Code*), uma camada de proteção que garante que o dado lido seja fiel ao dado gravado, independentemente de eventuais corrupções no meio físico.

3.1.2 Fundamentação Teórica: Do Bit de Paridade ao Código de Hamming

O Bit de Paridade Simples O mecanismo mais elementar de proteção de dados é o **bit de paridade**. A ideia consiste em adicionar um único bit extra a uma palavra de dados, de forma que o número total de bits em nível lógico ‘1’ seja sempre par (paridade par) ou sempre ímpar (paridade ímpar). Por exemplo, para a palavra de 4 bits 1011 (três bits em ‘1’), o bit de paridade par seria 1, resultando na palavra transmitida 1011 1.

O receptor recalcula a paridade e a compara com o bit recebido: se houver divergência, um erro de bit único é **detectado**. Contudo, o mecanismo possui duas limitações graves: (i) é incapaz de indicar *qual* bit foi corrompido, tornando a correção impossível; e (ii) é "cego" a erros duplos, pois dois bits invertidos simultaneamente cancelam-se na equação de paridade, mascarando a falha.

Distância de Hamming e Capacidade de Correção Richard Hamming resolveu essas limitações introduzindo o conceito de **Distância de Hamming** (d_{min}): o número mínimo de posições em que dois *codewords* válidos de um código diferem entre si. Esse valor determina matematicamente a capacidade do código:

- Para **detectar** até d erros, é necessário $d_{min} \geq d + 1$.
- Para **corrigir** até t erros, é necessário $d_{min} \geq 2t + 1$.

Um código de paridade simples possui $d_{min} = 2$ (detecta 1 erro, mas não corrige nada). Ao adicionar múltiplos bits de paridade organizados estrategicamente, Hamming conseguiu elevar d_{min} para 3, viabilizando a correção de 1 erro simples (SEC) e a detecção de até 2 erros (DED).

O Código de Hamming (7,4): Estrutura e Posicionamento O Hamming(7,4) codifica 4 bits de dados em uma palavra de 7 bits, utilizando 3 bits de paridade (p_1, p_2, p_4). O truque está no **posicionamento**: os bits de paridade ocupam exclusivamente as posições que são potências de 2 (1, 2, 4), enquanto os bits de dados preenchem as posições restantes (3, 5, 6, 7), conforme a Tabela 1.

Tabela 1: Posicionamento dos bits no codeword Hamming(7,4).

Posição	1	2	3	4	5	6	7
Bit	p_1	p_2	d_1	p_4	d_2	d_3	d_4

Cada bit de paridade cobre um subconjunto específico de posições, definido pela representação binária do índice da posição: p_1 cobre as posições cujo bit menos significativo é 1 (1, 3, 5, 7); p_2 cobre as posições cujo segundo bit é 1 (2, 3, 6, 7); e p_4 cobre as posições cujo terceiro bit é 1 (4, 5, 6, 7), conforme a Tabela 2.

Tabela 2: Cobertura de cada bit de paridade.

Paridade	Posições cobertas (XOR)
p_1	1, 3, 5, 7
p_2	2, 3, 6, 7
p_4	4, 5, 6, 7

Essa sobreposição deliberada é o que permite a **correção**: cada posição de dado é coberta por uma combinação única de bits de paridade, de modo que o padrão de divergências aponta exatamente qual bit falhou.

Exemplo Prático Completo Para ilustrar o funcionamento na prática, codificamos o nibble de dados $d_1d_2d_3d_4 = 1011$.

Codificação: calculamos os bits de paridade par (XOR das posições cobertas, conforme a Tabela 2):

$$p_1 = d_1 \oplus d_2 \oplus d_4 = 1 \oplus 0 \oplus 1 = 0$$

$$p_2 = d_1 \oplus d_3 \oplus d_4 = 1 \oplus 1 \oplus 1 = 1$$

$$p_4 = d_2 \oplus d_3 \oplus d_4 = 0 \oplus 1 \oplus 1 = 0$$

O codeword de 7 bits transmitido, seguindo a ordem posicional $p_1 p_2 d_1 p_4 d_2 d_3 d_4$, é: 0110011.

Injeção de erro: suponha que um evento de radiação inverta o bit na posição 6 (d_3), de 1 para 0. O codeword recebido passa a ser 0110 001 (posições 1 a 7).

Decodificação e cálculo da síndrome: o receptor recalcula cada paridade sobre o dado recebido. Uma divergência gera um bit de síndrome igual a 1:

$$c_1 = \text{pos}_1 \oplus \text{pos}_3 \oplus \text{pos}_5 \oplus \text{pos}_7 = 0 \oplus 1 \oplus 0 \oplus 1 = 0$$

$$c_2 = \text{pos}_2 \oplus \text{pos}_3 \oplus \text{pos}_6 \oplus \text{pos}_7 = 1 \oplus 1 \oplus \mathbf{0} \oplus 1 = 1$$

$$c_4 = \text{pos}_4 \oplus \text{pos}_5 \oplus \text{pos}_6 \oplus \text{pos}_7 = 0 \oplus 0 \oplus \mathbf{0} \oplus 1 = 1$$

A síndrome, lida como $c_4c_2c_1$, resulta em 110, que em decimal equivale a **6** — apontando **exatamente** a posição 6 como a localização do erro. A correção é trivial: basta inverter o bit naquela posição ($0 \rightarrow 1$), restaurando o codeword original 0110011 e, consequentemente, o nibble de dados correto 1011.

Esse mecanismo de síndrome — um valor numérico que aponta diretamente a posição da falha — é a base conceitual que o MRSC e o TBEC-RSC herdam e expandem, substituindo a busca por um único bit por um mapeamento de *regiões* inteiras da matriz de dados, adequado para o cenário mais agressivo dos *Multiple Cell Upsets*.

3.1.3 Fluxo Operacional de Codificação e Decodificação

O ciclo de vida de um dado protegido por ECC compreende duas fases críticas:

1. **Codificação (Escrita):** O processador envia o dado, e o controlador de memória calcula instantaneamente o código ECC correspondente. Ambos são salvos no barramento físico.
2. **Decodificação (Leitura):** Ao solicitar o dado, o controlador recalcula o ECC em tempo real. Se houver discrepância, o algoritmo identifica o bit corrompido e aplica a correção automaticamente antes de entregar a informação ao processador.

O custo dessa segurança reflete-se tradicionalmente em área de silício e latência. Entretanto, os mecanismos aqui propostos minimizam esse impacto por meio de uma infraestrutura 100% combinacional. Isso significa que o atraso inserido restringe-se apenas ao tempo de propagação intrínseco das portas lógicas (*gate-delay*), sem a necessidade de ciclos de clock adicionais, mantendo o desempenho em tempo real necessário para comunicações com memórias SRAM de alta velocidade.

3.2 Matrix Region Selection Code (MRSC)

3.2.1 Definição Teórica e Formulação Matemática

O Matrix Region Selection Code (MRSC) é um código de correção de erros bidimensional (2D) projetado para proteger memórias voláteis contra *Multiple Cell Upsets* (MCUs). Diferente de códigos tradicionais que utilizam polinômios complexos, o MRSC codifica 16 bits de dados em uma palavra de 32 bits valendo-se estritamente de operações lógicas simples de paridade (XOR), visando o mínimo impacto em hardware.

Os 16 bits de dados originais são divididos em quatro grupos de 4 bits, organizados em uma matriz (A, B, C, D) . A redundância de 16 bits gerada é composta por bits Diagonais (Di), de Paridade (P) e bits de Checagem (Cb). Esses bits são posicionados estrategicamente intercalados aos dados para aumentar a eficiência contra padrões de erros adjacentes.

Cálculo das Redundâncias e seus Papéis Cada subgrupo de redundância possui uma responsabilidade específica no algoritmo:

- **Bits Diagonais (Di) e Bits de Paridade (P):** São os principais responsáveis

pela **detecção da região falha**. Ao cruzar linhas, colunas e diagonais, eles geram síndromes que, quando avaliadas em conjunto, indicam em qual parte da matriz ocorreu a corrupção.

- **Bits de Checagem (Cb):** Atuam diretamente na **localização exata e correção**. Eles geram uma "máscara" que será aplicada exclusivamente sobre a região defeituosa identificada pelos bits Di e P .

As formulações matemáticas para a geração de cada bit redundante utilizam portas lógicas XOR ($\oplus$) e operam sobre os bits de dados (A_n, B_n, C_n, D_n) da seguinte forma:

1. **Bits Diagonais (Di):** Cruzam dados em diagonais específicas.

$$Di_1 = A_1 \oplus B_2 \oplus C_1 \oplus D_2$$

$$Di_2 = A_2 \oplus B_1 \oplus C_2 \oplus D_1$$

$$Di_3 = A_3 \oplus B_4 \oplus C_3 \oplus D_4$$

$$Di_4 = A_4 \oplus B_3 \oplus C_4 \oplus D_3$$

2. **Bits de Paridade (P):** Calculados nas colunas dos bits de dados.

$$P_1 = A_1 \oplus B_1 \oplus C_1 \oplus D_1$$

$$P_2 = A_2 \oplus B_2 \oplus C_2 \oplus D_2$$

$$P_3 = A_3 \oplus B_3 \oplus C_3 \oplus D_3$$

$$P_4 = A_4 \oplus B_4 \oplus C_4 \oplus D_4$$

3. **Bits de Checagem (Cb):** Calculados intercalando os bits (1, 3 e 2, 4) de cada grupo.

$$CbA_{13} = A_1 \oplus A_3$$

$$CbA_{24} = A_2 \oplus A_4$$

$$CbB_{13} = B_1 \oplus B_3$$

$$CbB_{24} = B_2 \oplus B_4$$

$$CbC_{13} = C_1 \oplus C_3$$

$$CbC_{24} = C_2 \oplus C_4$$

$$CbD_{13} = D_1 \oplus D_3$$

$$CbD_{24} = D_2 \oplus D_4$$

O fluxo de decodificação e correção do MRSC é executado em três etapas macro:

1. **Estimativa de Síndromes:** O circuito lê a palavra de 32 bits, recalcula internamente os bits de redundância (RDi, RP, RCb) a partir dos dados lidos e faz um

XOR com as redundâncias armazenadas originais. O resultado gera os vetores de síndrome SDi , SP e SCb .

2. **Verificação de Condição de Erro:** Um erro é flagrado se ocorrer uma de duas situações lógicas: (i) as síndromes SDi e SP tiverem, ambas, pelo menos um valor não nulo (diferente de zero); OU (ii) o somatório dos bits da síndrome SCb for maior que 1.
3. **Seleção da Região e Correção:** A matriz de dados é virtualmente sobreposta em 3 regiões (R_1, R_2, R_3). O algoritmo compara o somatório das síndromes SDi e SP das duas primeiras colunas contra as duas últimas colunas para determinar a região afetada, conforme a Tabela 3. Uma vez selecionada a região, o vetor SCb é submetido a uma operação XOR direta com os dados daquela região (aplicação da máscara) para inverter os bits corrompidos de volta ao estado original.

Tabela 3: Critério para Seleção de Região no MRSC.

Região Selecionada	Critério de Seleção Lógica
Região 1	$(SDi_1 + SDi_2 + P_1 + P_2) > (SDi_3 + SDi_4 + P_3 + P_4)$
Região 2	$(SDi_1 + SDi_2 + P_1 + P_2) < (SDi_3 + SDi_4 + P_3 + P_4)$
Região 3	$(SDi_1 + SDi_2 + P_1 + P_2) = (SDi_3 + SDi_4 + P_3 + P_4)$

3.2.2 Exemplo de Codificação e Decodificação

Para demonstrar o algoritmo MRSC na prática, utilizaremos o número 2003 (ano de fundação do LESC) como nosso dado bruto. O valor 2003 em binário de 16 bits é 0000 0111 1101 0011.

O primeiro passo da codificação é organizar esses 16 bits na matriz de dados 4×4 :

$$A = [A_1, A_2, A_3, A_4] = [0, 0, 0, 0]$$

$$B = [B_1, B_2, B_3, B_4] = [0, 1, 1, 1]$$

$$C = [C_1, C_2, C_3, C_4] = [1, 1, 0, 1]$$

$$D = [D_1, D_2, D_3, D_4] = [0, 0, 1, 1]$$

Passo 1: Geração da Redundância (Codificação) Aplicamos as fórmulas matemáticas do MRSC para gerar os bits Diagonais (Di), de Paridade (P) e de Checagem (Cb):

- **Paridade** ($P_x = A_x \oplus B_x \oplus C_x \oplus D_x$):

$$P_1 = 0 \oplus 0 \oplus 1 \oplus 0 = 1$$

$$P_3 = 0 \oplus 1 \oplus 0 \oplus 1 = 0$$

$$P_2 = 0 \oplus 1 \oplus 1 \oplus 0 = 0$$

$$P_4 = 0 \oplus 1 \oplus 1 \oplus 1 = 1$$

Vetor P gerado: 1001

- **Diagonal** ($Di_x = A_x \oplus B_y \oplus C_x \oplus D_y$):

$$Di_1 = A_1 \oplus B_2 \oplus C_1 \oplus D_2 = 0 \oplus 1 \oplus 1 \oplus 0 = 0$$

$$Di_2 = A_2 \oplus B_1 \oplus C_2 \oplus D_1 = 0 \oplus 0 \oplus 1 \oplus 0 = 1$$

$$Di_3 = A_3 \oplus B_4 \oplus C_3 \oplus D_4 = 0 \oplus 1 \oplus 0 \oplus 1 = 0$$

$$Di_4 = A_4 \oplus B_3 \oplus C_4 \oplus D_3 = 0 \oplus 1 \oplus 1 \oplus 1 = 1$$

Vetor Di gerado: 0101

- **Checagem** (Cb intercalado 1,3 e 2,4):

$$CbA = [0 \oplus 0, 0 \oplus 0] = [0, 0]$$

$$CbC = [1 \oplus 0, 1 \oplus 1] = [1, 0]$$

$$CbB = [0 \oplus 1, 1 \oplus 1] = [1, 0]$$

$$CbD = [0 \oplus 1, 0 \oplus 1] = [1, 1]$$

Vetor Cb gerado: 00 10 10 11

A palavra final de 32 bits codificada e gravada na memória é composta por: 0000 0111 1101 0011 [Dados] 0101 1001 [Di, P] 00101011 [Cb].

Passo 2: Injeção de Falha (MCU) Suponha que uma partícula ionizante atinja a memória e cause um erro duplo adjacente (MCU) na primeira coluna da matriz, invertendo os bits A_1 e B_1 de 0 para 1. Nossa matriz lida pelo decodificador agora contém erros:

$$A' = [1, 0, 0, 0]$$

$$B' = [1, 1, 1, 1]$$

Passo 3: Decodificação e Correção O circuito inicia o processo lendo a palavra de 32 bits e recalculando as redundâncias (RDi, RP, RCb) baseando-se exclusivamente nos dados recém-lidos (que agora contêm a falha: $A'_1 = 1$ e $B'_1 = 1$). Em seguida, uma operação lógica XOR é realizada entre essas novas redundâncias e as redundâncias originais armazenadas na memória para extrair os vetores de síndrome (SDi, SP, SCb).

O Ponto Cego da Paridade: Como o evento de múltipla falha (MCU) inverteu exatamente dois bits na mesma coluna (A_1 e B_1), o recálculo da paridade para a coluna 1 se anula matematicamente ($1 \oplus 1 = 0$). Isso resulta em um vetor de paridade recalculado idêntico ao original, gerando uma síndrome nula ($SP = 0000$). Se o sistema dependesse apenas de paridade, o erro passaria despercebido.

A Captura Transversal (SDi e SCb): Para contornar isso, os bits diagonais e de checagem cruzam a matriz de formas diferentes, isolando os bits corrompidos. O decodificador recalcula as redundâncias afetadas e extrai as síndromes que sinalizam o erro:

- **Síndrome Diagonal (SDi):** O recálculo cruza os dados corrompidos (A'_1 e B'_1).

$$RDi_1 = A'_1 \oplus B'_2 \oplus C'_1 \oplus D'_2 = 1 \oplus 1 \oplus 1 \oplus 0 = 1$$

$$RDi_2 = A'_2 \oplus B'_1 \oplus C'_2 \oplus D'_1 = 0 \oplus 1 \oplus 1 \oplus 0 = 0$$

O vetor recalculado é $RDi = 1001$. Ao compará-lo com o armazenado ($Di = 0101$):

$$SDi = Di \oplus RDi = 0101 \oplus 1001 = 1100$$

- **Síndrome de Checagem (SCb):** Avalia a relação intercalada das linhas.

$$SCbA_{13} = CbA_{13} \text{ original} \oplus (A'_1 \oplus A'_3) = 0 \oplus (1 \oplus 0) = 1$$

$$SCbB_{13} = CbB_{13} \text{ original} \oplus (B'_1 \oplus B'_3) = 1 \oplus (1 \oplus 1) = 1$$

Verificação da Condição de Erro: O algoritmo dita que a correção deve prosseguir se a soma de bits ativos (com valor 1) no vetor de síndrome SCb for maior que 1. Neste cenário, temos exatamente dois bits ativos ($SCbA_{13}$ e $SCbB_{13}$), o que aciona o gatilho de correção, provando que há um erro na matriz de dados.

Cálculo e Seleção de Região: Para descobrir em qual terço da matriz o erro ocorreu, aplicamos o critério da Tabela de Seleção. O algoritmo soma os bits individuais

das síndromes SDi e SP correspondentes à metade esquerda da matriz (colunas 1 e 2) e compara com a metade direita (colunas 3 e 4):

$$(SDi_1 + SDi_2 + SP_1 + SP_2) \quad \text{vs} \quad (SDi_3 + SDi_4 + SP_3 + SP_4)$$

Substituindo pelos valores obtidos ($SDi = \mathbf{1100}$ e $SP = \mathbf{0000}$):

$$(1 + 1 + 0 + 0) > (0 + 0 + 0 + 0) \implies 2 > 0$$

Como o somatório do lado esquerdo é estritamente maior, o algoritmo decreta matematicamente que a **Região 1** (que engloba as colunas 1 e 2) é o alvo da correção.

Correção: O vetor de síndrome SCb atua como a máscara de correção do circuito. Como a **Região 1** selecionada compreende as colunas 1 e 2 da matriz de dados, o algoritmo do MRSC mapeia os componentes de SCb diretamente sobre essas colunas: o primeiro componente de checagem (SCb_{13} , correspondente à relação entre os bits 1 e 3) dita a correção para a **coluna 1**, enquanto o segundo componente (SCb_{24} , correspondente à relação entre os bits 2 e 4) dita a correção para a **coluna 2**.

No cenário do nosso teste, o decodificador extrai as seguintes máscaras binárias $[SCb_{13}, SCb_{24}]$ para cada uma das linhas:

- **Linha A:** $SCbA = [SCbA_{13}, SCbA_{24}] = [\mathbf{1}, 0]$. Ao aplicar a operação XOR restritamente sobre os elementos da Região 1, o primeiro bit (**1**) inverte o dado da coluna 1 (A_1), corrigindo-o ($\mathbf{1} \oplus 1 = 0$), enquanto o segundo bit (**0**) mantém a coluna 2 (A_2) intacta ($0 \oplus 0 = 0$).

- **Linha B:** $SCbB = [SCbB_{13}, SCbB_{24}] = [1, 0]$. De forma idêntica, o bit ativo (**1**) força a inversão lógica do dado contido na coluna 1 (B_1), restabelecendo o valor correto ($1 \oplus 1 = 0$).

Com a aplicação desse mapeamento posicional, os bits afetados pelo MCU retornam ao seu estado original de maneira limpa e isolada, validando a capacidade do MRSC de contornar falhas simultâneas que silenciariam sistemas baseados em paridade convencional.

3.2.3 Análise de Vantagens e Desvantagens

Vantagens: A principal força do MRSC é seu baixíssimo custo de implementação, o que o torna ideal para aplicações críticas com restrições severas de recursos. Em sínteses de 65 nm, seu decodificador demonstrou consumir aproximadamente apenas $59 \mu\text{W}$ de potência e menor área/atraso em comparação a algoritmos mais densos (como o Reed-Muller 2,5 ou o CLC), pois não faz uso de pesadas tabelas de correspondência (*matchup tables*) para localizar os erros.

Desvantagens: O calcanhar de aquiles do MRSC reside na sua dependência estrutural em 2D. Implementar sua arquitetura em memórias físicas requer o fatiamento da palavra em múltiplos endereços, gerando gargalos de acesso. Além disso, embora seja excelente para corrigir erros adjacentes (vizinhos), o algoritmo sofre forte queda de eficácia ao se deparar com erros triplos espaçados (ex: padrões de falha 101 ou 111), limitação que motivou a evolução posterior para o TBEC-RSC.

3.3 Triple Burst Error Correction Based on Region Selection Code (TBEC-RSC)

3.3.1 Definição Teórica e Formulação Matemática

O *Triple Burst Error Correction Based on Region Selection Code* (TBEC-RSC) surge como uma evolução direta do MRSC, concebido para mitigar os gargalos físicos de implementação e sanar falhas críticas de cobertura do seu antecessor. O TBEC-RSC mantém a filosofia de baixo custo computacional (utilizando apenas lógica XOR), mas

reestrutura a codificação de uma matriz 2D para um formato de palavra linear 1D.

Essa planificação estrutural resolve o maior problema prático do MRSC: a necessidade de fatiar a palavra em múltiplos endereços de memória, o que gerava atrasos de leitura e escrita. Com o TBEC-RSC, os 16 bits de dados e os 16 bits de redundância formam um bloco contínuo de 32 bits, gravado e lido em um único ciclo de acesso na memória.

Para preservar a eficiência na localização de falhas adjacentes, o TBEC-RSC mantém a lógica matemática exata do MRSC para a geração dos **Bits Diagonais** (Di) e dos **Bits de Checagem** (Cb):

$$Di_x = A_x \oplus B_y \oplus C_x \oplus D_y \quad (\text{onde } x \in \{1, 2, 3, 4\} \text{ e } y \in \{2, 1, 4, 3\}) \quad (1)$$

$$CbA_{vw} = A_v \oplus A_w \quad (\text{intercalando } v \in \{1, 2\} \text{ e } w \in \{3, 4\} \text{ para todos os grupos}) \quad (2)$$

A grande revolução matemática do TBEC-RSC ocorre na formulação dos **Bits de Paridade** (P). No MRSC original, a paridade era calculada verticalmente (isolada por colunas). Isso criava um ponto cego letal: erros triplos *burst* não-adjacentes (como os padrões 101 ou 111), ao atingirem dois bits da mesma coluna, anulavam a equação de paridade e passavam despercebidos pelo sistema.

O TBEC-RSC conserta essa falha agrupando a paridade de forma mista, cruzando elementos de linhas e colunas diferentes para garantir que erros espaçados não se anulem. As novas equações utilizam os índices $x \in \{1, 3\}$ e $y \in \{2, 4\}$:

$$P_x = A_x \oplus A_y \oplus B_x \oplus B_y \quad (3)$$

$$P_y = C_x \oplus C_y \oplus D_x \oplus D_y \quad (4)$$

O fluxo de decodificação baseia-se na extração das mesmas síndromes originais (SDi, SP, SCb) por meio da operação XOR entre as redundâncias recalculadas e as armazenadas. No entanto, o MRSC possuía uma segunda vulnerabilidade: ao lidar com padrões severos de erros triplos adjacentes que afetavam apenas os bits de redundância, o algoritmo "enxergava" um falso erro nos dados e executava uma correção indevida, corrompendo informações válidas.

Para impedir essas falsas correções, o decodificador do TBEC-RSC introduz um filtro de segurança antes de autorizar o prosseguimento da correção. O sistema avalia a

seguinte **Condição Especial**:

- A síndrome SDi é totalmente nula.
- A síndrome SP possui pelo menos um bit ativo (igual a 1).
- A síndrome SCb possui dois ou mais bits ativos.

Se todas essas premissas forem verdadeiras, o algoritmo deduz matematicamente que o dano ocorreu exclusivamente na redundância e aborta a correção, preservando os dados intactos. Caso a condição retorne falsa, o algoritmo segue para a verificação padrão de seleção, utilizando os mesmos critérios e a mesma divisão de regiões herdados do MRSC:

Tabela 4: Critério para Seleção de Região no TBEC-RSC.

Região Selecionada	Critério de Seleção Lógica
Região 1	$\sum_{i=1}^2 SDi_i + SP_i > \sum_{i=3}^4 SDi_i + SP_i$
Região 2	$\sum_{i=1}^2 SDi_i + SP_i < \sum_{i=3}^4 SDi_i + SP_i$
Região 3	$\sum_{i=1}^2 SDi_i + SP_i = \sum_{i=3}^4 SDi_i + SP_i$

4 Requisitos do Projeto e Ambiente de Teste

4.1 Especificações de Hardware e Placa Alvo

A plataforma de validação escolhida para a implementação física do multiplexador reconfigurável de ECC é a FPGA **SmartFusion2 M2S025-VFG256**, da família Microsemi (atualmente Microchip), encapsulada em pacote FBGA de 256 pinos. Todo o fluxo de projeto – descrição RTL em SystemVerilog, síntese lógica, mapeamento físico (*place and route*) e geração do arquivo de configuração (*bitstream*) – é conduzido através do **Libero SoC**, o ambiente de desenvolvimento proprietário do fabricante, que integra em um único fluxo as etapas de *front-end* e *back-end* necessárias para gravar o design na FPGA.

A FPGA opera como um elemento intermediário transparente entre um processador STM32, que atua como mestre do sistema, e duas memórias SRAM externas, que armazenam a informação redundante gerada pelo ECC. A comunicação entre o STM32 e a FPGA é realizada através do barramento **FMC** (*Flexible Memory Controller*) nativo do microcontrolador, configurado para operar com palavras de 16 bits. O módulo `fpga_top_design` (Nível 3, Seção 5.3) expõe exatamente essa fronteira física em suas portas: um barramento de dados bidirecional de 16 bits voltado ao processador (`mcu_fpga_io`), dois barramentos de dados bidirecionais de 16 bits voltados às memórias (`fpga_mem_io_up` e `fpga_mem_io_down`), os sinais de controle `write_en`, `chip_sel` e `output_en`, o seletor de algoritmo `ecc_sel` de 3 bits, o sinal de saída `chip_sel_out` de 2 bits (um por memória) e o vetor de diagnóstico `flag_out`, que reporta ao processador se e onde uma correção foi aplicada.

A numeração física de cada um desses sinais nos pinos da FPGA e nos GPIOs do STM32 está documentada em uma planilha de referência cruzada, mantida separadamente do código-fonte. Para que o sistema funcione corretamente após a gravação do projeto na FPGA, o firmware do STM32 deve configurar todos os GPIOs envolvidos como saída (OUTPUT) com resistor de *pull-down* interno habilitado, e dois desses GPIOs devem ser reservados exclusivamente para acionar o seletor `ecc_sel`, permitindo a troca dinâmica entre os quatro modos de operação implementados no Nível 2 (Seção 5.2).

4.2 Cenários de Validação e Requisitos de Teste

A validação do projeto ocorre em duas frentes complementares. A primeira é puramente simulada, empregando *testbenches* não sintetizáveis (Seção 2) que exercitam individualmente cada um dos quatro modos selecionáveis por `ecc_sel`: *bypass* direto na Memória 1, *bypass* direto na Memória 2, ECC TBEC-RSC e ECC MRSC. Para os dois modos com correção de erro, o ambiente de simulação injeta artificialmente padrões de falha representativos dos efeitos de radiação discutidos na Seção 1: *bit-flips* isolados (SEU), pares de bits adjacentes invertidos na mesma coluna da matriz de dados (MCU) e, criticamente, padrões de erro triplo espaçado (como 101 e 111) que caracterizam o ponto cego identificado na Seção 3. Este último cenário é o requisito de teste mais relevante do projeto: é justamente a condição em que se espera observar a falha de correção do

MRSC e a correção bem-sucedida pelo TBEC-RSC, comprovando experimentalmente a motivação teórica para a evolução entre os dois algoritmos. Adicionalmente, os testes de simulação devem cobrir o **filtro de segurança** do decodificador TBEC-RSC (Seção 5.1), forçando cenários em que apenas os bits de redundância são corrompidos, para confirmar que o circuito *não* executa uma correção indevida sobre dados válidos.

A segunda frente de validação ocorre diretamente no hardware físico. Com o projeto gravado na FPGA e o STM32 operando via FMC, o requisito de teste consiste em realizar ciclos de escrita e leitura reais nas duas memórias SRAM, alternando o seletor `ecc_sel` através dos GPIOs mapeados, e comparando o dado lido de volta pelo processador com o dado originalmente escrito. Para os modos com ECC, o teste de aceitação exige ainda que o vetor `flag_out` seja observado: ele deve permanecer nulo em operações sem falha e deve indicar corretamente a região corrigida (bit1, bit2 ou bit3) quando uma corrupção é forçada manualmente em uma das memórias, sem gerar falsos positivos durante execuções de referência (*golden runs*) livres de falha.

5 Arquitetura do Hardware e Implementação (Abordagem Bottom-Up)

Esta seção descreve a implementação em SystemVerilog do multiplexador de ECC seguindo uma metodologia **bottom-up**: parte-se dos blocos lógicos mais elementares — os encoders e decoders de cada algoritmo de correção — para, em seguida, demonstrar como esses blocos são integrados progressivamente em módulos de hierarquia superior, até compor o sistema completo.

A arquitetura do projeto é organizada em três níveis hierárquicos, conforme ilustrado na Figura 1:

1. **Nível 1 – Codecs individuais:** Os módulos `MRSC_encoder`, `MRSC_decoder`, `TBEC_RSC_encoder` e `TBEC_RSC_decoder` implementam isoladamente a lógica de codificação e decodificação de cada algoritmo, conforme as formulações matemáticas apresentadas na Seção 3. Cada codec opera de forma puramente combinacional, convertendo uma palavra de 16 bits de dados em um *codeword* de 32 bits (encoders) ou realizando o

caminho inverso com correção de erros (decoders).

2. **Nível 2 – Multiplexador de ECC (ecc_design):** Este módulo instancia simultaneamente os quatro codecs do Nível 1 e utiliza um sinal de seleção (**sel**) para escolher, em tempo real, o modo de operação do fluxo de dados entre o processador e as duas memórias SRAM externas: aplicação do algoritmo **TBEC-RSC**, aplicação do algoritmo **MRSC**, ou modo *bypass* (encaminhamento direto dos dados, sem qualquer codificação, para a memória 1 ou para a memória 2), totalizando quatro modos de operação distintos.
3. **Nível 3 – Módulo Top-Level (fpga_top_design):** Camada mais externa do projeto, responsável por instanciar o **ecc_design** e gerenciar a interface física do chip: os barramentos bidirecionais (**inout**) conectados ao microcontrolador e às duas memórias, o controle dos buffers *tri-state* que arbitram o acesso a esses barramentos compartilhados, e o registrador síncrono de flags de erro (**flag_out**).

Nas subseções seguintes, cada um desses três níveis é detalhado individualmente, iniciando pelos codecs MRSC e TBEC-RSC.

5.1 Nível 1: Codecs Individuais

5.1.1 Codificador e Decodificador MRSC

Codificador (MRSC_encoder) O módulo **MRSC_encoder** recebe uma palavra de 16 bits (**data_in**) e produz um *codeword* de 32 bits (**data_out**), aplicando as equações de D_i , P e C_b apresentadas na Seção 3.

```
1 module MRSC_encoder(input logic [0:15] data_in, output logic [0:31]
  data_out);
2   logic [0:3] signal [0:3]; // matriz 4x4:signal[grupo][posicao]
3   logic Di[0:3];
4   logic P[0:3];
5   logic [0:1] Cb[0:3];
6
7   always_comb
8   begin
```

```

9      integer b; // Inteiro
10     // Divisao dos 16 bits de entrada nos 4 grupos (A, B, C, D)
11     signal[0][0:3]=data_in[0:3];    // Grupo A
12     signal[1][0:3]=data_in[4:7];    // Grupo B
13     signal[2][0:3]=data_in[8:11];   // Grupo C
14     signal[3][0:3]=data_in[12:15];  // Grupo D
15
16     // Bits de checagem Cb: XOR intercalado (posicoes 183 e 284 de
17     // cada grupo)
18     for(b=0;b<4;b=b+1)
19     begin
20         Cb[b][0]=signal[b][0]^signal[b][2];
21         Cb[b][1]=signal[b][1]^signal[b][3];
22     end
23
24     // Bits de paridade P: XOR vertical entre os 4 grupos, coluna a
25     // coluna
26     P[0]=signal[0][0]^signal[1][0]^signal[2][0]^signal[3][0];
27     P[1]=signal[0][1]^signal[1][1]^signal[2][1]^signal[3][1];
28     P[2]=signal[0][2]^signal[1][2]^signal[2][2]^signal[3][2];
29     P[3]=signal[0][3]^signal[1][3]^signal[2][3]^signal[3][3];
30
31     // Bits diagonais Di: XOR cruzado entre grupos adjacentes
32     Di[0]=signal[0][0]^signal[1][1]^signal[2][0]^signal[3][1];
33     Di[1]=signal[0][1]^signal[1][0]^signal[2][1]^signal[3][0];
34     Di[2]=signal[0][2]^signal[1][3]^signal[2][2]^signal[3][3];
35     Di[3]=signal[0][3]^signal[1][2]^signal[2][3]^signal[3][2];
36 end
37
38 // Montagem do codeword de 32 bits: dados reorganizados +
39 // redundancia
40 assign data_out={signal[0][0],signal[1][0],signal[2][0],signal
41                 [3][0],
42                 signal[0][1],signal[1][1],signal[2][1],signal[3][1],

```

```

39     signal[0][2], signal[1][2], signal[2][2], signal[3][2],
40     signal[0][3], signal[1][3], signal[2][3], signal[3][3],
41     Di[0], Di[3], Di[1], Di[2], P[0], P[3], P[1], P[2],
42     Cb[0][0:1], Cb[1][0:1], Cb[2][0:1], Cb[3][0:1]};
43 endmodule

```

Listing 9: Codificador MRSC.

O código segue fielmente as três famílias de equações do MRSC:

- **Divisão em grupos (linhas 11–14):** os 16 bits de entrada são fatiados nos quatro grupos de 4 bits (A, B, C, D), armazenados na matriz `signal[grupo][posição]`.
- **Bits de checagem Cb (laço for):** para cada grupo, calcula-se o XOR entre as posições 1–3 e 2–4, implementando diretamente as equações $CbX_{13} = X_1 \oplus X_3$ e $CbX_{24} = X_2 \oplus X_4$.
- **Bits de paridade P e diagonais Di :** calculados exatamente como nas equações originais, cruzando os quatro grupos coluna a coluna (P) ou em diagonal (Di).
- **Montagem do `data_out` (linha assign):** aqui está a principal adaptação de implementação. Em vez de armazenar os 16 bits de dados na ordem original ($A_1A_2A_3A_4B_1B_2B_3B_4\dots$), o código os reorganiza **transpondo a matriz**: os quatro primeiros bits do *codeword* são A_1, B_1, C_1, D_1 (a primeira coluna da matriz 4×4), os quatro seguintes são A_2, B_2, C_2, D_2 , e assim por diante. Essa reorganização é o que permite ao MRSC ser armazenado em um **único endereço de memória de 32 bits**, eliminando a limitação original do artigo de 2017, que exigia 4 endereços separados para a forma matricial 2-D. Os 16 bits finais concatenam os bits Di, P e Cb (em uma ordem intercalada $[Di_1, Di_4, Di_2, Di_3]$ e $[P_1, P_4, P_2, P_3]$, escolhida para manter a proximidade física entre bits de redundância relacionados).

Decodificador (MRSC_decoder) O módulo `MRSC_decoder` recebe o *codeword* de 32 bits e devolve os 16 bits de dados corrigidos, executando as três etapas descritas na Seção 3: cálculo de síndromes, verificação da condição de erro e seleção/correção de região.

```

1 module MRSC_decoder(input logic [0:31] data_in, output logic [0:15]
    data_out);

```

```

2 logic [0:3] signal [0:3];
3 logic [0:1] linsin [0:3]; // SCb: síndrome dos bits de checagem
4 logic [0:3] SP;           // síndrome de paridade
5 logic [0:3] SDi;          // síndrome diagonal
6
7 always_comb
8 begin
9     integer b;
10    integer sumSL, sumSP, sumSDi;
11    integer quad1;
12    integer quad2;
13
14    // Desfaz a transposicao: reconstroi os grupos A, B, C, D
    originais
15    signal[0][0:3]={data_in[0],data_in[4],data_in[8],data_in[12]};
        // A
16    signal[1][0:3]={data_in[1],data_in[5],data_in[9],data_in[13]};
        // B
17    signal[2][0:3]={data_in[2],data_in[6],data_in[10],data_in[14]};
        // C
18    signal[3][0:3]={data_in[3],data_in[7],data_in[11],data_in[15]};
        // D
19
20    // SCb: recalcula Cb sobre os dados lidos e compara (XOR) com o
    Cb armazenado
21    linsin[0][0]=signal[0][0]^signal[0][2]^data_in[24];
22    linsin[0][1]=signal[0][1]^signal[0][3]^data_in[25];
23    linsin[1][0]=signal[1][0]^signal[1][2]^data_in[26];
24    linsin[1][1]=signal[1][1]^signal[1][3]^data_in[27];
25    linsin[2][0]=signal[2][0]^signal[2][2]^data_in[28];
26    linsin[2][1]=signal[2][1]^signal[2][3]^data_in[29];
27    linsin[3][0]=signal[3][0]^signal[3][2]^data_in[30];
28    linsin[3][1]=signal[3][1]^signal[3][3]^data_in[31];
29

```

```

30      // SP: recalcula P sobre os dados lidos e compara com o P
      armazenado
31      SP[0]=signal[0][0]^signal[1][0]^signal[2][0]^signal[3][0]^
data_in[20];
32      SP[1]=signal[0][1]^signal[1][1]^signal[2][1]^signal[3][1]^
data_in[22];
33      SP[2]=signal[0][2]^signal[1][2]^signal[2][2]^signal[3][2]^
data_in[23];
34      SP[3]=signal[0][3]^signal[1][3]^signal[2][3]^signal[3][3]^
data_in[21];
35
36      // SDi: recalcula Di sobre os dados lidos e compara com o Di
      armazenado
37      SDi[0]=signal[0][0]^signal[1][1]^signal[2][0]^signal[3][1]^
data_in[16];
38      SDi[1]=signal[0][1]^signal[1][0]^signal[2][1]^signal[3][0]^
data_in[18];
39      SDi[2]=signal[0][2]^signal[1][3]^signal[2][2]^signal[3][3]^
data_in[19];
40      SDi[3]=signal[0][3]^signal[1][2]^signal[2][3]^signal[3][2]^
data_in[17];
41
42      // Soma total dos 8 bits de SCb (usada na condicao ii do artigo
      )
43      sumSL = 1*linsin[1][0]+1*linsin[1][1]+1*linsin[2][0]+1*linsin
[2][1]
44      + 1*linsin[3][0]+1*linsin[3][1]+1*linsin[0][0]+1*linsin
[0][1];
45
46      // Condicao de erro: (SP!=0 E SDi!=0) OU (soma de SCb > 1)
47      if((((SP[0:3]!=4'b0000)&(SDi[0:3]!=4'b0000)) | sumSL>1))
48      begin
49          // Soma das sindromes das colunas 1-2 vs colunas 3-4
50          quad1=1*SP[0]+1*SP[1]+1*SDi[0]+1*SDi[1];

```

```

51     quad2=1*SP [2]+1*SP [3]+1*SDi [2]+1*SDi [3];
52
53     if(quad1 > quad2) // Regiao 1: corrige colunas 1-2
54     begin
55         for(b=0;b<4;b=b+1)
56             if(linsin[b][0:1]!=2'b00)
57                 signal[b][0:1]=signal[b][0:1]^linsin[b][0:1];
58     end
59     else if(quad1 < quad2) // Regiao 2: corrige colunas 3-4
60     begin
61         for(b=0;b<4;b=b+1)
62             if(linsin[b][0:1]!=2'b00)
63                 signal[b][2:3]=signal[b][2:3]^linsin[b][0:1];
64     end
65     else if((quad1==quad2)&({SP[0],SP[1],SDi[0],SDi[1]}!=4'b0000)) // Regiao 3
66     begin
67         for(b=0;b<4;b=b+1)
68             if(linsin[b][0:1]!=2'b00)
69                 signal[b][1:2]=signal[b][1:2]^{linsin[b][1],
70                 linsin[b][0]};
71     end
72     end
73 assign data_out={signal[0][0:3],signal[1][0:3],signal[2][0:3],
74                 signal[3][0:3]};
75 endmodule

```

Listing 10: Decodificador MRSC.

A decodificação segue exatamente as três etapas do algoritmo original:

- **Reconstrução dos grupos (linhas 11–14):** como o encoder transpôs a matriz na montagem do data_out, o decoder faz o processo inverso, reagrupando os bits nas posições 0, 4, 8, 12 (grupo *A*), 1, 5, 9, 13 (grupo *B*), e assim sucessivamente.

- **Cálculo das síndromes ($linsin$, SP , SDi):** cada síndrome é obtida recalculando a equação de redundância correspondente sobre os dados lidos e aplicando XOR com o valor armazenado no *codeword* (extraído das posições fixas 16–31). A variável *linsin* corresponde à síndrome SCb do artigo; o nome sugere “síndrome de linha”.
- **Verificação da condição de erro:** a linha do *if* implementa exatamente as duas condições do artigo – $SP!=0 \ \&\& \ SDi!=0$ (ambas as síndromes possuem ao menos um bit ativo) $|| \ sumSL>1$ (dois ou mais bits de SCb ativos).
- **Seleção de região e correção:** *quad1* e *quad2* somam as síndromes das colunas 1–2 e 3–4, respectivamente, implementando o critério da Tabela 3. A correção final aplica XOR entre os dados e a síndrome SCb (*linsin*) apenas na região identificada como corrompida – observe que, na Região 3, os bits de *linsin* são invertidos de ordem ($\{linsin[b][1], linsin[b][0]\}$), reproduzindo exatamente o deslocamento (*shift*) descrito na Figura 3(b) do artigo original.

5.1.2 Codificador e Decodificador TBEC-RSC

Codificador (TBEC_RSC_encoder) O módulo *TBEC_RSC_encoder* partilha a mesma infraestrutura base do codificador MRSC, uma vez que a disposição do *codeword* em memória e o cálculo da maior parte das redundâncias se mantêm. A principal alteração reflete a atualização teórica do cálculo de paridade para mitigar erros triplos espaçados.

```

1 module TBEC_RSC_encoder( input logic [0:15] data_in,
2                           output logic [0:31] data_out
3                           );
4
5     logic [0:3] signal [0:3]; // Matriz 4x4: signal[grupo][posicao]
6     logic Di[0:3];           // Bits Diagonais
7     logic P[0:3];            // Bits de Paridade
8     logic [0:1] Cb[0:3];      // Bits de Checagem
9     always_comb
10 begin
11     integer b;
12     // Divisao dos 16 bits de entrada nos 4 grupos (A, B, C, D)
13     signal[0][0:3]=data_in[0:3]; // Grupo A

```

```

14  signal[1][0:3]=data_in[4:7];    // Grupo B
15  signal[2][0:3]=data_in[8:11];  // Grupo C
16  signal[3][0:3]=data_in[12:15]; // Grupo D
17
18  // Bits de checagem Cb: XOR intercalado (posicoes 183 e 284 de
    // cada grupo)
19  for(b=0;b<4;b=b+1)
20  begin
21      Cb[b][0]=signal[b][0]^signal[b][2];
22      Cb[b][1]=signal[b][1]^signal[b][3];
23  end
24
25  // Bits de paridade P: XOR cruzado (mitigacao de erros triplos
    // espacados)
26  P[0]=signal[0][0]^signal[0][1]^signal[1][0]^signal[1][1];
27  P[1]=signal[2][0]^signal[2][1]^signal[3][0]^signal[3][1];
28  P[2]=signal[0][2]^signal[0][3]^signal[1][2]^signal[1][3];
29  P[3]=signal[2][2]^signal[2][3]^signal[3][2]^signal[3][3];
30
31  // Bits diagonais Di: XOR cruzado entre grupos adjacentes
32  Di[0]=signal[0][0]^signal[1][1]^signal[2][0]^signal[3][1];
33  Di[1]=signal[0][1]^signal[1][0]^signal[2][1]^signal[3][0];
34  Di[2]=signal[0][2]^signal[1][3]^signal[2][2]^signal[3][3];
35  Di[3]=signal[0][3]^signal[1][2]^signal[2][3]^signal[3][2];
36  end
37  // Montagem do codeword de 32 bits: dados reorganizados +
    // redundancia
38  assign data_out={signal[0][0],signal[1][0],signal[2][0],signal
    [3][0],
39      signal[0][1],signal[1][1],signal[2][1],signal[3][1],
40      signal[0][2],signal[1][2],signal[2][2],signal[3][2],
41      signal[0][3],signal[1][3],signal[2][3],signal[3][3],
42      Di[0],Di[3],Di[1],Di[2],P[0],P[3],P[1],P[2],Cb[0][0:1],Cb
    [1][0:1],Cb[2][0:1],Cb[3][0:1]};

```

```

43
44
45 endmodule

```

Listing 11: Codificador TBEC-RSC.

A análise do código RTL evidencia os seguintes pontos operacionais:

- **Cálculo Misto de Paridade (Linhas 21–24):** Ao contrário da paridade puramente vertical do MRSC, as equações agrupam bits adjacentes de linhas distintas (ex: $P[0]$ agrupa as colunas 0 e 1 das linhas A e B). Esta transposição a nível de portas lógicas assegura que os padrões de erro não-adjacentes sejam intercetados por cruzamentos matemáticos diferentes, impossibilitando o cancelamento mútuo da síndrome.
- **Preservação Estrutural:** O método de fatiamento dos dados, o cálculo dos bits de checagem (Cb) e das diagonais (Di), bem como a transposição final para o vetor de 32 bits (linha `assign`), permanecem idênticos ao MRSC, o que garante a coesão arquitetural do sistema ao multiplexar ambos os códigos.

Decodificador (TBEC_RSC_decoder) O módulo de decodificação acomoda não só as novas fórmulas de paridade, mas também implementa ativamente o filtro de segurança (Condição Especial) para impedir a corrupção de dados por falsos positivos em cenários em que apenas a redundância foi afetada. Adicionalmente, introduz a sinalização externa do erro.

```

1 module TBEC_RSC_decoder(input logic[0:31] data_in, output logic
  [0:15] data_out, output logic [0:2] flag);
2 logic [0:3] signal [0:3]; // Matriz 4x4
3 logic [0:15] red;          // Vetor isolado para as redundancias
  armazenadas
4 logic [0:1] linsin [0:3]; // SCb: síndrome dos bits de checagem
5 logic [0:3] SP;           // Síndrome de paridade
6 logic [0:3] SDi;          // Síndrome diagonal
7 logic bit1;               // Flag para erro na Regiao 1
8 logic bit2;               // Flag para erro na Regiao 2

```

```

9  logic bit3;                                // Flag para erro na Regiao 3
10
11  always_comb
12  begin
13
14      integer b;
15      integer sumSL, sumSP, sumSDi;
16      integer quad1;
17      integer quad2;
18
19      // Desfaz a transposicao: reconstroi os grupos A, B, C, D
20      // originais
21      signal[0][0:3]={data_in[0], data_in[4], data_in[8], data_in[12]};
22      signal[1][0:3]={data_in[1], data_in[5], data_in[9], data_in[13]};
23      signal[2][0:3]={data_in[2], data_in[6], data_in[10], data_in[14]};
24      signal[3][0:3]={data_in[3], data_in[7], data_in[11], data_in[15]};
25
26      // Extrai as redundancias lidas para o vetor 'red'
27      red[0:15]=data_in[16:31];
28
29      // SCb: recalcula Cb sobre os dados lidos e compara (XOR) com o
30      // Cb armazenado
31      linsin[0][0]=signal[0][0]^signal[0][2]^red[8];
32      linsin[0][1]=signal[0][1]^signal[0][3]^red[9];
33      linsin[1][0]=signal[1][0]^signal[1][2]^red[10];
34      linsin[1][1]=signal[1][1]^signal[1][3]^red[11];
35      linsin[2][0]=signal[2][0]^signal[2][2]^red[12];
36      linsin[2][1]=signal[2][1]^signal[2][3]^red[13];
37      linsin[3][0]=signal[3][0]^signal[3][2]^red[14];
38      linsin[3][1]=signal[3][1]^signal[3][3]^red[15];
39
40      // SP: recalcula P (formato cruzado do TBEC) e compara com o P
41      // armazenado
42      SP[0]=signal[0][0]^signal[0][1]^signal[1][0]^signal[1][1]^red[4];

```

```

40 SP[1]=signal[2][0]^signal[2][1]^signal[3][0]^signal[3][1]^red[6];
41 SP[2]=signal[0][2]^signal[0][3]^signal[1][2]^signal[1][3]^red[7];
42 SP[3]=signal[2][2]^signal[2][3]^signal[3][2]^signal[3][3]^red[5];
43 sumSP=1*SP[0] + 1*SP[1] + 1*SP[2] + 1*SP[3]; // Soma de bits da
    síndrome de paridade
44
45 // SDi: recalcula Di e compara com o Di armazenado
46 SDi[0]=signal[0][0]^signal[1][1]^signal[2][0]^signal[3][1]^red
    [0];
47 SDi[1]=signal[0][1]^signal[1][0]^signal[2][1]^signal[3][0]^red
    [2];
48 SDi[2]=signal[0][2]^signal[1][3]^signal[2][2]^signal[3][3]^red
    [3];
49 SDi[3]=signal[0][3]^signal[1][2]^signal[2][3]^signal[3][2]^red
    [1];
50 sumSDi=1*SDi[0] + 1*SDi[1] + 1*SDi[2] + 1*SDi[3]; // Soma de bits
    da síndrome diagonal
51
52 // Soma total dos 8 bits de SCb
53 sumSL=1*linsin[1][0] + 1*linsin[1][1]+ 1*linsin[2][0]+ 1*linsin
    [2][1]+1*linsin[3][0] + 1*linsin[3][1]+ 1*linsin[0][0]+ 1*linsin
    [0][1];
54
55 // Condicao para deteccao de erros nos dados (inclui filtro de
    seguranca para evitar falsa correcao)
56 if((((SP[0:3] !=4'b0000) & (SDi[0:3] !=4'b0000)) | sumSL>1) &
    !((sumSDi==0) & (sumSP==1) & (sumSL>=2)))// condicao para erros
    em quadrantes
57 begin
58     // Soma das síndromes para selecao da regioao
59     quad1=1*SP[0]+1*SP[1]+1*SDi[0]+1*SDi[1];
60     quad2=1*SP[2]+1*SP[3]+1*SDi[2]+1*SDi[3];
61
62     if(quad1 > quad2)// Erro no primeiro quadrante (Regiao 1)

```

```

63     begin
64         bit1 = 1;
65         for(b=0;b<4;b=b+1)
66             begin
67                 if(linsin[b][0:1]!=2'b00)
68                     signal[b][0:1]=signal[b][0:1]^linsin[b][0:1]; // Aplica
mascara SCb nas colunas 1 e 2
69             end
70         end
71
72         else if(quad1 < quad2)// Erro no segundo quadrante (Regiao 2)
73         begin
74             bit2 = 1;
75             for(b=0;b<4;b=b+1)
76                 begin
77                     if(linsin[b][0:1]!=2'b00)
78                         signal[b][2:3]=signal[b][2:3]^linsin[b][0:1]; // Aplica
mascara SCb nas colunas 3 e 4
79                 end
80             end
81
82         else if((quad1 == quad2)&({SP[0],SP[1],SDi[0],SDi[1]} !=4'b0000
83 )) // Erro no quadrante central (Regiao 3)
84         begin
85             bit3 = 1;
86             for(b=0;b<4;b=b+1)
87                 begin
88                     if(linsin[b][0:1]!=2'b00)
89                         signal[b][1:2]=signal[b][1:2]^{linsin[b][1],linsin[b]
90 ] [0]}; // Aplica mascara com shift
91                 end
92             end
93         else

```

```

93         begin
94             bit1 = 0;
95             bit2 = 0;
96             bit3 = 0;
97         end
98     end
99 end
100 assign data_out={signal[0][0:3],signal[1][0:3],signal[2][0:3],
    signal[3][0:3]};
101 assign flag = {bit1,bit2,bit3};
102 endmodule

```

Listing 12: Decodificador TBEC-RSC.

A decodificação do TBEC-RSC expande as capacidades de correção mediante os seguintes aspetos construtivos fundamentais:

- **Extração de Redundância Isolada:** Em vez de extrair os bits da redundância de forma estática no decorrer das equações (como o MRSC fazia), o decodificador atribui explicitamente as posições 16 a 31 do barramento `data_in` ao vetor dedicado `red[0:15]`, conferindo maior organização à lógica combinacional de reavaliação matemática.
- **Filtro de Segurança (Linha 48):** A adição do termo booleano `& !((sumSDi==0) & (sumSP==1) & (sumSL>=2))` traduz diretamente para hardware a Condição Especial teórica (Secção 3.3.1). Caso esta cláusula detete que as síndromes provêm exclusivamente de danos na zona das redundâncias, toda a operação de correção no bloco `if` é impedida, travando corrupções induzidas por uma sobrecorreção.
- **Geração de Sinalização (Flags):** Ao contrário do seu antecessor, o módulo declara o vetor de saída `flag[0:2]`. Durante a secção condicional de seleção da matriz, o decodificador ativa individualmente as lógicas combinacionais internas (`bit1`, `bit2` ou `bit3`). Esta funcionalidade é crucial na arquitetura global do Multiplexador (Nível 3), dado que as flags são conduzidas para a hierarquia de Top-Level a fim de alertarem processadores externos acerca da taxa e da região dos erros corrigidos em tempo real.

5.2 Nível 2: Multiplexador de ECC (ecc_design)

Enquanto o Nível 1 trata da matemática pura dos códigos de correção, o Nível 2 (módulo `ecc_design`) é o núcleo de roteamento da arquitetura. A sua função é instanciar os codificadores e decodificadores construídos anteriormente e, com base num sinal de controlo externo (`sel`), decidir que caminho o fluxo de dados deve seguir entre o processador e as duas memórias SRAM externas.

```
1 'include "TBEC_RSC_encoder.sv"
2 'include "TBEC_RSC_decoder.sv"
3 'include "MRSC_encoder.sv"
4 'include "MRSC_decoder.sv"
5
6 module ecc_design(
7     input  logic [0:15] data_in_left,      // Dado bruto vindo do
8         processor (FPGA In)
9     output logic [0:15] data_out_right_up,  // Via de dados para a
10        Memoria 1
11     output logic [0:15] data_out_right_down, // Via de dados para a
12        Memoria 2
13     input  logic [0:15] data_in_right_up,   // Dado lido da Memoria
14        1
15     input  logic [0:15] data_in_right_down, // Dado lido da Memoria
16        2
17     output logic [0:15] data_out_left,      // Dado processado/
18        corrigido (FPGA Out)
19     input  logic [2:0] sel,                  // Seletor de modo de
20        operacao (ECC ou Bypass)
21     output logic [0:2] flag,                 // Sinalizacao de erro
22        exportada
23     output logic [1:0] chip_sel_out,         // Habilitacao
24        individual das memorias (CS)
25     input  logic      chip_sel               // Sinal de habilitacao
26        mestre
27 );
```

```

18
19 // Fios internos para interligar os encoders/decoders ao
    multiplexador
20 logic [0:15] decoder_output1;
21 logic [0:31] encoder_output1;
22 logic [0:15] decoder_output2;
23 logic [0:31] encoder_output2;
24
25 logic [0:15] decoder_output3;
26 logic [0:31] encoder_output3;
27 logic [0:15] decoder_output4;
28 logic [0:31] encoder_output4;
29
30 // Instanciacao dos Codecs (Bottom-Up)
31 // TBEC-RSC (ECC 1)
32 TBEC_RSC_encoder codificador1 (
33     .data_in(data_in_left),
34     .data_out(encoder_output1)
35 );
36 // O decoder recebe a concatenacao de 32 bits das duas memorias
    de 16 bits
37 TBEC_RSC_decoder decodificador1 (
38     .data_in({data_in_right_up, data_in_right_down}),
39     .data_out(decoder_output1),
40     .flag(flag)
41 );
42
43 // MRSC (ECC 2)
44 MRSC_encoder codificador2 (
45     .data_in(data_in_left),
46     .data_out(encoder_output2)
47 );
48 MRSC_decoder decodificador2 (
49     .data_in({data_in_right_up, data_in_right_down}),

```

```

50     .data_out(decoder_output2)
51 );
52
53 // Bloco Combinacional de Multiplexacao e Roteamento
54 always_comb begin
55     // Inicializacao padrao para prevenir a criacao de latches
indesejados
56     data_out_right_up    = '0;
57     data_out_right_down = '0;
58     data_out_left       = '0;
59     chip_sel_out        = 2'b11;
60
61     // Selecao dinamica do caminho de dados
62     case (sel)
63         3'b000 : begin // BYPASS: Comunicacao direta com a Memoria
1 (Sem ECC)
64             data_out_right_up = data_in_left;
65             data_out_left     = data_in_right_up;
66             chip_sel_out      = {chip_sel, !chip_sel}; // Habilita
apenas Mem 1
67         end
68
69         3'b001 : begin // BYPASS: Comunicacao direta com a Memoria
2 (Sem ECC)
70             data_out_right_down = data_in_left;
71             data_out_left       = data_in_right_down;
72             chip_sel_out        = {!chip_sel, chip_sel}; //
Habilita apenas Mem 2
73         end
74
75         3'b010 : begin // MODO ECC 1: Algoritmo TBEC-RSC
76             // 0 codeword de 32 bits e fatiado: metade para a Mem
1, metade para a Mem 2
77             data_out_right_up    = encoder_output1[0:15];

```

```

78         data_out_right_down = encoder_output1[16:31];
79         // O dado ja corrigido pelo decodificador e repassado a
saída
80         data_out_left      = decoder_output1;
81         // Ambas as memorias sao ativadas simultaneamente para
gravar/ler 32 bits
82         chip_sel_out      = {chip_sel, chip_sel};
83     end
84
85     3'b011 : begin // MODO ECC 2: Algoritmo MRSC
86         data_out_right_up  = encoder_output2[0:15];
87         data_out_right_down = encoder_output2[16:31];
88         data_out_left      = decoder_output2;
89         chip_sel_out      = {chip_sel, chip_sel};
90     end
91 endcase
92 end
93 endmodule

```

Listing 13: Lógica estrutural e de roteamento do Multiplexador de ECC.

A estrutura RTL deste módulo centraliza o fluxo de informações (*datapath*) explorando várias técnicas essenciais de descrição de hardware:

- **Instanciação Paralela e Concatenação:** As instâncias codificador e decodificador operam de forma ininterrupta e paralela. Uma particularidade fundamental deste Nível 2 ocorre na leitura dos dados: os barramentos das duas memórias SRAM de 16 bits (*data_in_right_up* e *data_in_right_down*) são fundidos através do operador de concatenação {}, reconstruindo fisicamente o *codeword* contínuo de 32 bits exigido pelas portas *data_in* dos decodificadores.
- **Prevenção de *Latches*:** Em conformidade com a teoria apresentada na Secção 2, o bloco *always_comb* inicia-se com uma atribuição de valor neutro ('0') a todas as portas de saída. Esta prática assegura que, perante modos de operação indefinidos no *case*, nenhuma saída retenha o seu estado anterior, prevenindo a síntese de

latches na FPGA.

- **Modos *Bypass* vs Modos ECC:** A lógica de habilitação (`chip_sel_out`) revela o comportamento físico do sistema. Nos modos *Bypass* (000 e 001), a operação de memória é de 16 bits, logo, o controlador inverte o sinal de *Chip Select* para garantir que apenas uma das SRAMs é ativada. Já nos modos protegidos por ECC (010 e 011), o sistema necessita armazenar 32 bits. Nesse caso, a diretiva `chip_sel_out = {chip_sel, chip_sel}` desperta simultaneamente ambas as memórias SRAM, paralelizando a escrita para manter a operação em tempo real (ciclo único de processamento).

5.3 Nível 3: Módulo Top-Level (`fpga_top_design`)

O Nível 3 constitui a camada mais externa da arquitetura, atuando como o invólucro físico (Top Design) do multiplexador. A sua função transcende o processamento lógico dos algoritmos de ECC; ele é o responsável direto pela gestão elétrica dos pinos da FPGA e pela arbitragem da comunicação bidirecional com os periféricos externos (o microcontrolador STM32 e os dois bancos de memória SRAM).

É neste módulo que os conceitos de portas bidirecionais (`inout`) e *buffers tri-state*, abordados na Secção 2, são aplicados na prática para evitar colisões de dados nos barramentos partilhados.

```
1 'include "ecc_design.sv"
2
3 module fpga_top_design(
4     inout wire [15:0] mcu_fpga_io,          // Barramento bidirecional
5     inout wire [15:0] fpga_mem_io_up,      // Barramento bidirecional
6     inout wire [15:0] fpga_mem_io_down,    // Barramento bidirecional
7     input logic      write_en,             // Sinal de habilitacao de
8     input logic      chip_sel,             // Sinal mestre de selecao
```

```

do chip (Chip Select)
9   output logic [1:0] chip_sel_out,      // Controle individual de
selecao das duas memorias
10  input logic [2:0] ecc_sel,           // Seletor de modo de
operacao do ECC
11  output logic [0:2] flag_out,         // Saida registrada das
flags de erro
12  input logic      clk,                // Relogio do sistema (
Clock)
13  input logic      output_en          // Sinal de habilitacao de
saida (Output Enable)
14 );

15
16  // Fios logicos internos para roteamento de dados bidirecionais
17  logic [15:0] encoder_input;
18  logic [15:0] decoder_output;
19  logic [15:0] encoder_output_up;
20  logic [15:0] decoder_input_up;
21  logic [15:0] encoder_output_down;
22  logic [15:0] decoder_input_down;
23  logic [0:2] flag;
24
25  // -----
26  // 1. Controle do Buffer Tri-state: Interface MCU-FPGA
27  // -----
28  // Se for leitura (write_en == 0 e memoria habilitada), a FPGA
envia o dado corrigido.
29  // Caso contrario, o pino fica em alta impedancia ('z') para o
MCU poder escrever.
30  assign mcu_fpga_io = (write_en == 0 && (chip_sel_out [1] == 0
|| chip_sel_out[0] == 0)) ? 16'bz : decoder_output;
31  assign encoder_input = mcu_fpga_io; // A entrada do codificador
escuta permanentemente o pino
32

```

```

33 // -----
34 // 2. Controle dos Buffers Tri-state: Interface FPGA-Memorias
35 // -----
36
37 // Escrita na Memoria 1 (UP): Ativa o driver se write_en=1 e
chip_sel_out[1]=0
38 assign fpga_mem_io_up =
39     (write_en==1'b1 && chip_sel_out[1]==1'b0) ?
encoder_output_up : 16'bz;
40
41 // Escrita na Memoria 2 (DOWN): Ativa o driver se write_en=1 e
chip_sel_out[0]=0
42 assign fpga_mem_io_down =
43     (write_en==1'b1 && chip_sel_out[0]==1'b0) ?
encoder_output_down : 16'bz;
44
45 // Leitura da Memoria 1 (UP): Desliga o driver interno (Alta
impedancia) na leitura
46 assign decoder_input_up =
47     (write_en==1'b0 && chip_sel_out[1]==1'b0) ? 16'bz :
fpga_mem_io_up;
48
49 // Leitura da Memoria 2 (DOWN): Desliga o driver interno (Alta
impedancia) na leitura
50 assign decoder_input_down =
51     (write_en==1'b0 && chip_sel_out[0]==1'b0) ? 16'bz :
fpga_mem_io_down;
52
53 // -----
54 // 3. Logica Sequencial: Registro das Flags de Erro
55 // -----
56 always_ff @(posedge clk) begin
57     // Durante a escrita, reseta as flags (sem avaliacao de
erro)

```

```

58         if (write_en == 0 && (chip_sel_out [1] == 0 || chip_sel_out
[0] == 0)) begin
59             flag_out <= 3'b000;
60         end
61         // Durante a leitura (output_en=0 e memorias ativas),
registra as falhas detectadas
62         else if (output_en == 0 && (chip_sel_out [1] == 0 ||
chip_sel_out[0] == 0)) begin
63             flag_out <= flag;
64         end
65         // Caso as memorias estejam inativas, mantem o estado
anterior
66         else begin
67             flag_out <= flag_out;
68         end
69     end

70
71     // -----
72     // 4. Instanciacao do Multiplexador (Nivel 2)
73     // -----
74     ecc_design modulo(
75         .data_in_left(encoder_input),           // Entrada vinda
do processador
76         .data_out_right_up(encoder_output_up),   // Saida para a
memoria 1
77         .data_out_right_down(encoder_output_down), // Saida para a
memoria 2
78         .data_in_right_up(decoder_input_up),     // Entrada lida
da memoria 1
79         .data_in_right_down(decoder_input_down), // Entrada lida
da memoria 2
80         .data_out_left(decoder_output),          // Saida
processada para o MCU
81         .sel(ecc_sel),                           // Sinal de

```

```

82      .flag(flag),                                // Flags internas
      dos decodificadores
83      .chip_sel_out(chip_sel_out),                // Habilitacao
      exportada para as memorias
84      .chip_sel(chip_sel)                          // Sinal de
      selecao primario
85  );
86
87  endmodule

```

Listing 14: Módulo estrutural Top-Level do multiplexador (`fpga_top_design`).

A consolidação do Top-Level evidencia três frentes estruturais indispensáveis:

- **Arbitragem por Alta Impedância ('Z'):** Através da atribuição contínua com operadores ternários (`? :`), o sistema gere de forma estrita o estado dos fios `inout`. Quando o processador externo pretende realizar uma escrita (`write_en = 1`), a FPGA isola eletricamente a via de saída (`mcu_fpga_io = 16'bz`) para atuar apenas como ouvinte, canalizando a informação recebida para o submódulo `ecc_design`. O mesmo princípio aplica-se à condução para as memórias, evitando curtos-circuitos elétricos.
- **Gestão Síncrona de Flags (`always_ff`):** Ao passo que todo o processamento de correção de erros ocorre em lógica combinacional pura, o registo de sinalização de erro à saída da FPGA (`flag_out`) baseia-se num bloco sequencial associado ao flanco de subida do relógio (`posedge clk`). Isto assegura que a CPU externa apenas detete sinais estáveis e defina prioridades distintas entre os ciclos de leitura e escrita.
- **Encerramento Hierárquico (*Bottom-Up*):** A secção final do código conclui a metodologia construtiva do hardware: o módulo instanciado (`ecc_design`) liga todos os fios lógicos internos, encapsulando os Níveis 1 e 2 num bloco monolítico e robusto, pronto a ser sintetizado fisicamente na matriz lógica da FPGA.

5.4 Fluxos Críticos de Sincronismo

5.4.1 Fluxo de Operação de Escrita (*Write Flow*)

Durante o ciclo de escrita, o microcontrolador (dispositivo mestre) inicia a transação colocando o seu próprio barramento em estado de saída, injetando o dado bruto e ativando o sinal de controle `write_en = 1`. O módulo Top-Level reage instantaneamente colocando o *buffer tri-state* do pino `mcu_fpga_io` em estado de alta impedância (`16'bz`), passando a atuar estritamente como receptor.

O dado de 16 bits flui de forma contínua para o módulo `ecc_design`, onde atravessa a lógica combinacional do codificador selecionado (MRSC ou TBEC-RSC). No tempo correspondente ao atraso de propagação das portas lógicas (*gate-delay*), o *codeword* de 32 bits é montado. Em seguida, o Top-Level habilita os seus *drivers* de saída em direção à SRAM. O vetor é fatiado: a metade superior é enviada à Memória 1 via `fpga_mem_io_up` e a metade inferior à Memória 2 via `fpga_mem_io_down`, ocorrendo a gravação paralela e síncrona nos dois chips físicos.

5.4.2 Fluxo de Operação de Leitura (*Read Flow*)

O ciclo de leitura exige uma inversão completa no controle de posse dos barramentos. O microcontrolador altera o estado para `write_en = 0` e recolhe os seus *drivers*. Ao detectar esta mudança, o Top-Level desativa os *buffers* conectados às memórias, permitindo que as SRAMs externas tomem posse dos fios físicos e conduzam os 32 bits armazenados para dentro da FPGA.

O *codeword* fracionado é concatenado na entrada do `ecc_design` e processado pelo decodificador. O algoritmo recalcula as paridades, extrai as síndromes e, caso os dados tenham sofrido *upsets* por radiação, aplica ativamente as máscaras de correção. O dado de 16 bits, agora restaurado e íntegro, é conduzido para a via `decoder_output`. Por fim, o Top-Level aciona o *buffer* do barramento principal para entregar a informação limpa ao microcontrolador. Simultaneamente, caso o sinal `output_en` permita, as falhas flagradas pelo decodificador são salvas no registrador síncrono `flag_out` na borda de subida do *clock* (`posedge clk`), disponibilizando o alerta para o tratamento de interrupções do firmware.

5.5 Diagrama de Blocos Estrutural

A Figura 1 consolida visualmente a hierarquia de roteamento descrita nas subseções anteriores. O "Controle Externo" governa as operações injetando os sinais de configuração (*chip select*, *write enable*) diretamente no "Top Multiplexador".

Este módulo atua como a ponte direcional central: isola eletricamente as vias bidirecionais (*Inout*) e distribui o *datapath* para os núcleos de processamento puramente combinacionais (instâncias ECC 1 e ECC 2). O fluxo resultante deságua no banco de memórias, evidenciando o paralelismo arquitetural construído para sustentar a largura de banda de 32 bits exigida pelos algoritmos espaciais, mantendo a operação transparente para a CPU.

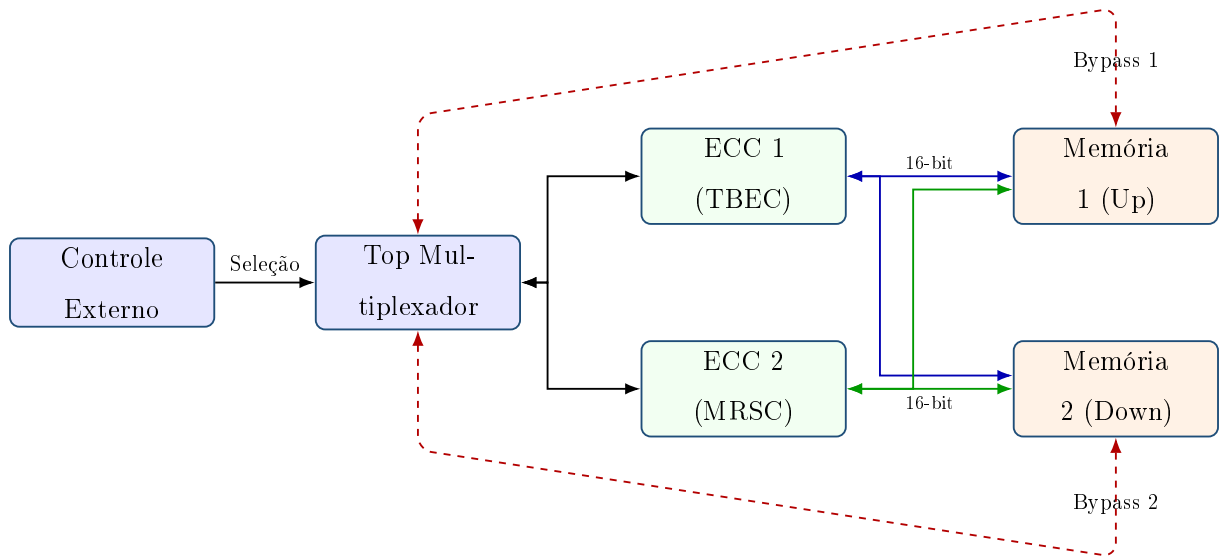


Figura 1: Fluxo de barramento integrado com roteamento bidirecional (inout) independente para as SRAMs.

6 Controle Externo e Interface com Periféricos

6.1 Integração com Microcontrolador (STM32)

O STM32 desempenha o papel de **firmware supervisor** do sistema, sendo o único elemento com conhecimento de mais alto nível sobre quando e por que trocar o algoritmo de proteção ativo – por exemplo, em resposta a telemetria de radiação, a uma

janela de maior criticidade da missão, ou a uma decisão de economia de energia, conforme discutido na motivação do projeto (Seção 1). Do ponto de vista elétrico, essa decisão se traduz apenas na escrita de níveis lógicos em GPIOs configurados como saída com *pull-down*: dois deles compõem o barramento `ecc_sel` reconhecido pelo Nível 2 (`ecc_design`), enquanto outros correspondem a `write_en`, `chip_sel` e `output_en`, sinais que o firmware manipula diretamente para orquestrar os fluxos de escrita e leitura descritos na Seção 5.3.

O acesso aos dados propriamente ditos ocorre através do periférico FMC do STM32, configurado para operar como uma memória externa de 16 bits. Isso permite ao firmware realizar leituras e escritas na região de memória mapeada com instruções convencionais de acesso à memória, sem necessidade de um driver de comunicação dedicado – toda a complexidade de codificação e correção permanece encapsulada e invisível para o software. O único retorno explícito de diagnóstico ao firmware é o vetor `flag_out`, registrado sincronamente no Top-Level e disponível ao STM32 para eventual tratamento de interrupção ou telemetria de taxa de erro.

6.2 Interface com Memória Externa (SRAM)

As duas memórias SRAM externas são conectadas cada uma a um dos barramentos bidirecionais de 16 bits da FPGA (`fpga_mem_io_up` e `fpga_mem_io_down`) e operam sob o controle direto dos sinais `chip_sel_out` e `write_en`, conforme detalhado no fluxo de escrita e leitura da Seção 5.3. O endereçamento físico dessas memórias é resolvido externamente pelo periférico FMC do STM32, cabendo à FPGA apenas garantir que a memória correta esteja habilitada (*chip select*) e que o sentido do tráfego de dados esteja coerente com o restante do sistema – a arquitetura não expõe um barramento de endereço próprio em suas portas.

Nos modos de *bypass* (`sel = 000` ou `001`), apenas uma memória de 16 bits é acessada por vez, e o `chip_sel_out` correspondente inverte um dos seus bits para isolar a memória inativa. Já nos modos protegidos por ECC (`sel = 010` ou `011`), ambas as memórias são habilitadas simultaneamente (`chip_sel_out = {chip_sel, chip_sel}`), pois o *codeword* de 32 bits gerado pelo codificador é fisicamente fatiado entre as duas SRAMs, cada uma recebendo metade da palavra em um único ciclo de acesso. Por se tratar de uma infraestrutura inteiramente combinacional (Seção 3), o único acréscimo de

latência introduzido pela codificação e decodificação do ECC é o atraso de propagação intrínseco (*gate-delay*) das portas lógicas envolvidas, preservando a compatibilidade com os tempos de ciclo exigidos por SRAMs de alta velocidade e evitando qualquer penalidade adicional de *clock* no acesso à memória.

7 Resultados e Considerações Finais

A arquitetura apresentada nesta documentação foi implementada e validada em nível de simulação por meio dos *testbenches* descritos na Seção 4. Os resultados obtidos demonstram que os algoritmos MRSC e TBEC-RSC apresentam o comportamento esperado para os cenários de teste propostos, evidenciando as capacidades e limitações de cada esquema de correção de erros conforme discutido ao longo deste documento.

A organização da arquitetura em três níveis hierárquicos mostrou-se adequada para separar as responsabilidades do sistema. O Nível 1 concentra a implementação dos codecs de ECC, o Nível 2 é responsável pela seleção do algoritmo e pelo roteamento dos dados, enquanto o Nível 3 realiza a integração com a memória SRAM e com os sinais externos da FPGA. Essa estrutura modular facilita a compreensão do funcionamento do sistema, bem como sua manutenção e reutilização.

A implementação utiliza uma arquitetura predominantemente combinacional, mantendo apenas o registrador de sinalização descrito na Seção 5.3. Essa abordagem reduz a complexidade do projeto e permite que os dois algoritmos de ECC sejam selecionados em tempo de execução por meio do sinal de controle, sem necessidade de reconfiguração da FPGA.

Dessa forma, a documentação apresenta de forma completa a arquitetura proposta, sua organização em módulos, o funcionamento de cada nível da hierarquia, a integração entre os componentes e o processo de validação realizado, servindo como referência para implementação, manutenção e utilização do sistema.

8 Tutorial: Gravação do Bitstream na FPGA com o Libero SoC 2025.2

Esta seção apresenta, em formato de tutorial passo a passo, o fluxo prático utilizado para abrir o projeto FPGA2memory no **Libero SoC Design Suite v2025.2** e gravar (*program*) o bitstream resultante na FPGA **SmartFusion2 M2S025T-VFG256**, fechando o ciclo entre a descrição RTL apresentada nas Seções 5 e 6 e sua materialização física no hardware.

8.1 Abertura do Projeto no Libero SoC

Ao iniciar o Libero SoC v2025.2, a ferramenta apresenta a *Welcome Page*, contendo atalhos para criar um novo projeto ou abrir um projeto existente, além de links de documentação da suíte (Figura 2).

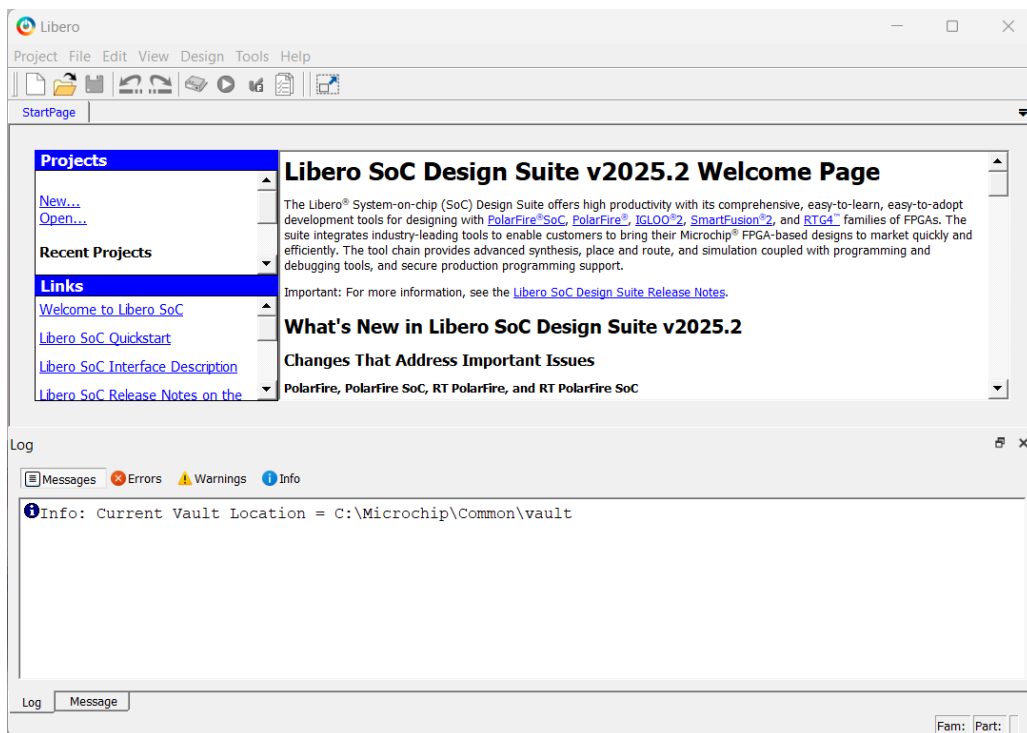


Figura 2: Página de boas-vindas do Libero SoC Design Suite v2025.2.

A partir do menu **Project** → **Open Project** (ou pelo link *Open...* na própria *Welcome Page*), navega-se até a pasta raiz do repositório extraído e seleciona-se o arquivo de projeto com extensão **.prjx**, localizado dentro da subpasta **FPGA2memory** (Figura 3). É

esse arquivo que referencia toda a estrutura interna do projeto – pastas `hdl`, `constraint`, `synthesis`, `designer`, entre outras.

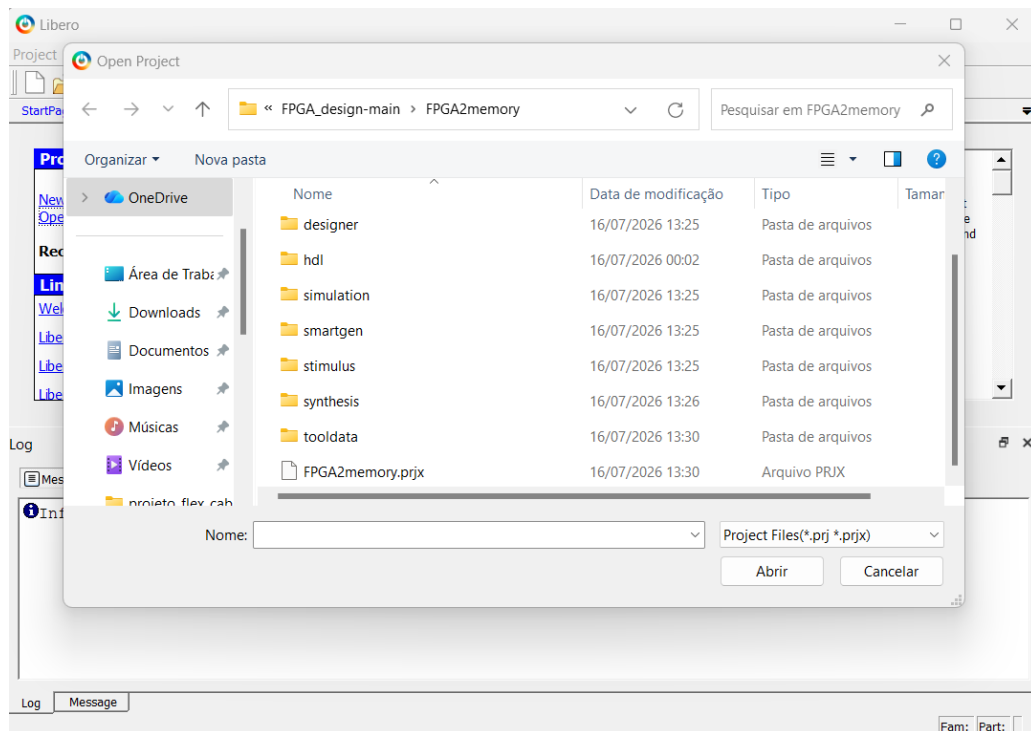


Figura 3: Seleção do arquivo de projeto `FPGA2memory.prjx` na janela *Open Project*.

8.2 Síntese do Design

Com o projeto aberto, o **Design Flow** (painel à esquerda) exibe todas as etapas do fluxo, da simulação pré-síntese até a gravação final. A primeira etapa executada é a **Synthesize**, que invoca o motor de síntese *Synplify Pro* para converter a descrição RTL em SystemVerilog (Seção 5) em uma *netlist* de portas lógicas mapeada para a arquitetura da SmartFusion2 (Figura 4).

O log da ferramenta *Synplify Pro* é exibido em detalhe na aba **Reports**, reportando a versão do sintetizador, o caminho de instalação e os argumentos de linha de comando utilizados na chamada interna do `synbatch.exe` (Figura 5). Ao final dessa etapa, o Libero indica *"Synthesis completed"* no painel de mensagens.

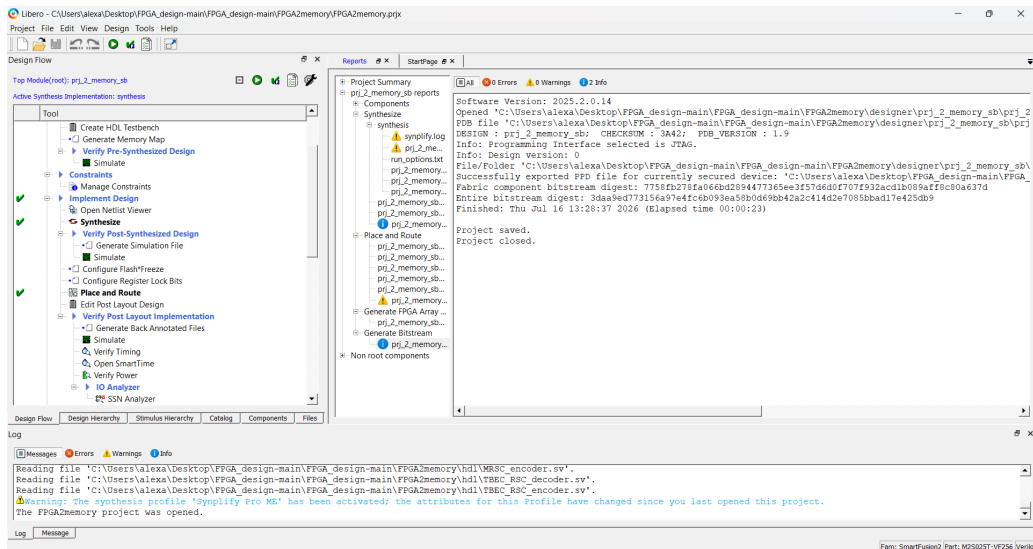


Figura 4: Árvore do Design Flow durante a leitura dos arquivos-fonte (MRSC_encoder.sv, TBEC_RSC_decoder.sv, etc.) na etapa de síntese.

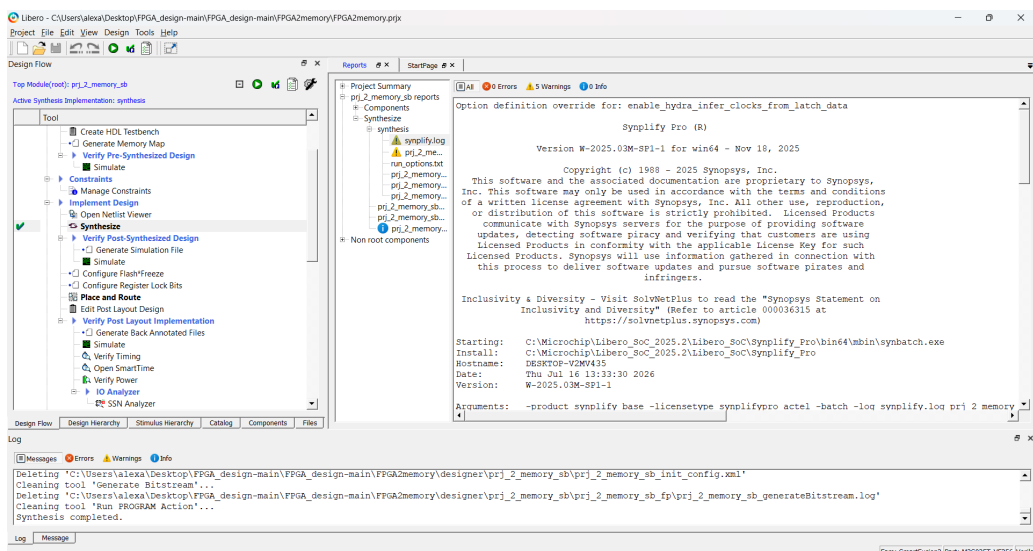


Figura 5: Log de execução do Synplify Pro (v2025.03M-SP1-1) durante a síntese do projeto.

8.3 Place and Route

Concluída a síntese, a próxima etapa do fluxo é o **Place and Route**, responsável por mapear fisicamente a *netlist* sintetizada nas células lógicas, blocos de I/O e recursos de *clock* disponíveis na FPGA M2S025T. Ao término dessa etapa, o Libero disponibiliza diversos relatórios técnicos – entre eles o **Global Net Report**, que detalha a alocação das redes de *clock* global (*Global Buffers - GB*) e sua taxa de *fanout* (Figura 6).

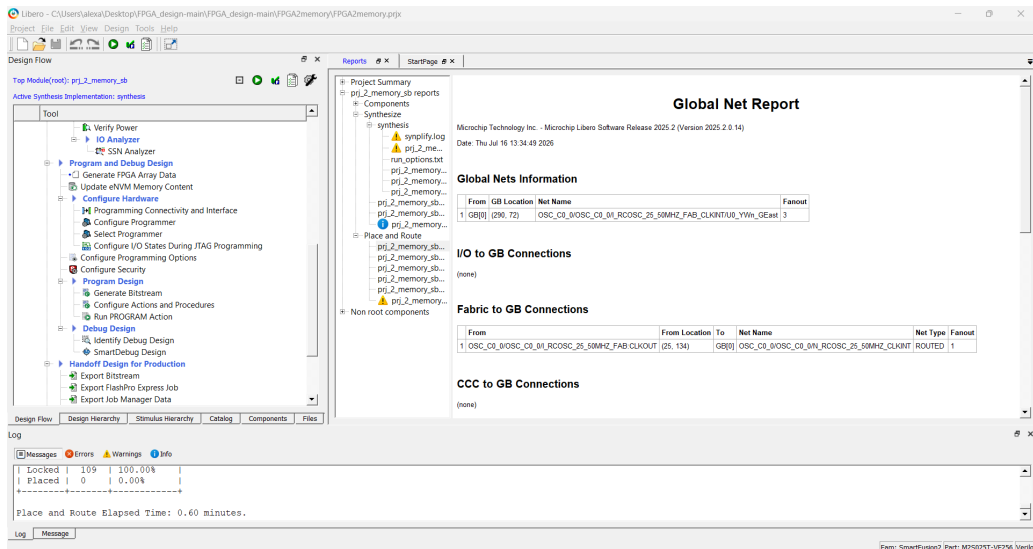


Figura 6: Relatório de redes globais (*Global Net Report*) gerado após o Place and Route.

8.4 Geração do Bitstream

Com o *place and route* concluído com sucesso (indicado pelo ícone de checagem verde ao lado da etapa na árvore do Design Flow), executa-se a etapa **Generate Bitstream**, dentro da seção **Program Design**. Essa etapa compila o arquivo de configuração binário que será efetivamente transferido para a FPGA. O log confirma a finalização do processo com a mensagem *"Generating Bitstream File Finished"*, acompanhada dos *digests* criptográficos do bitstream gerado, utilizados para verificação de integridade (Figura 7).

8.5 Seleção do Programador (FlashPro) e Gravação

Com a placa de desenvolvimento conectada ao computador via cabo USB-JTAG (programador **FlashPro5**), acessa-se a etapa **Select Programmer**, dentro de **Configure Hardware**. O Libero identifica automaticamente os programadores conectados fisicamente ao barramento USB, exibindo seu identificador de série, o tipo (**FlashPro5**) e a porta de comunicação (Figura 8). Essa etapa é crítica: o identificador do programador salvo no arquivo de projeto deve corresponder ao programador fisicamente conectado no momento da gravação – caso contrário, o Libero reporta que o programador esperado não pôde ser detectado.

Após confirmar o programador correto, executa-se a etapa final **Run PROGRAM**

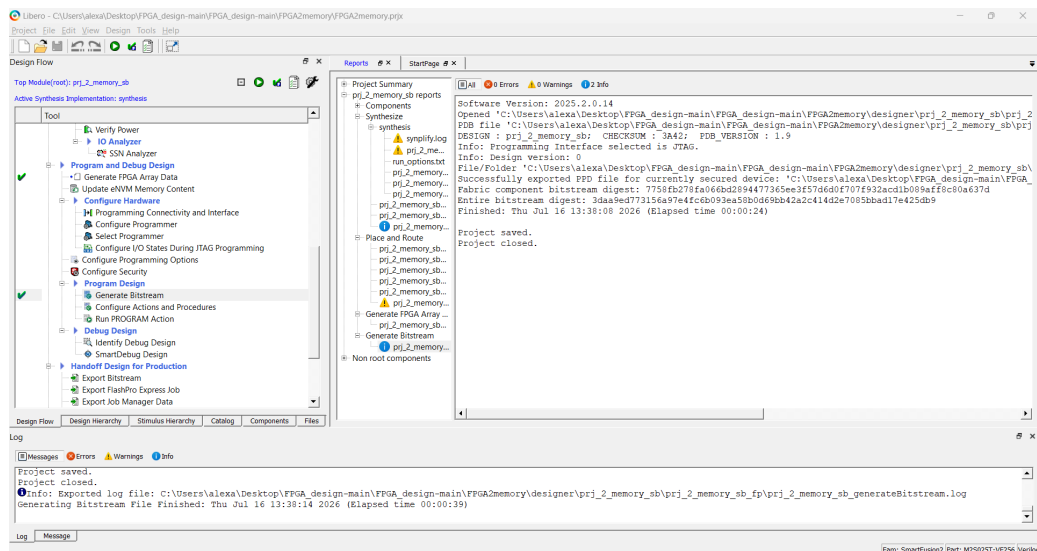


Figura 7: Conclusão da etapa Generate Bitstream, com os digests do fabric e do bitstream completo.

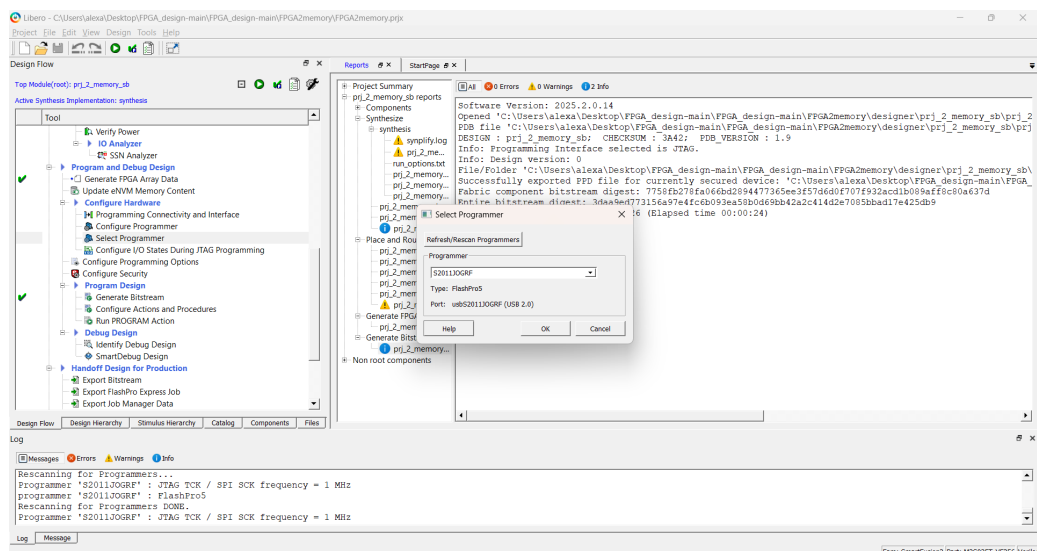


Figura 8: Janela *Select Programmer* detectando o FlashPro5 conectado (porta USB 2.0).

Action. O Libero verifica a cadeia JTAG (*Check Chain*), confirma que o dispositivo físico corresponde ao *part number* configurado no projeto (M2S025T-VF256) e, em caso positivo, transfere o bitstream gerado na Seção 8 para a FPGA. Ao final da operação bem-sucedida, o projeto é salvo e fechado automaticamente pela ferramenta (Figura 9).

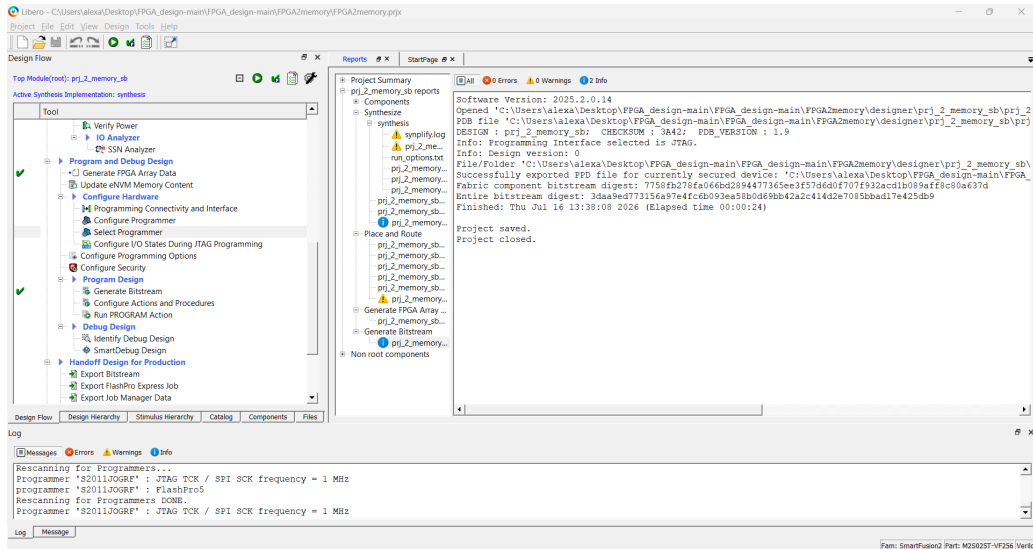


Figura 9: Conclusão do fluxo de programação: projeto salvo e fechado após a gravação bem-sucedida do bitstream na FPGA.

8.6 Resumo do Fluxo

Em síntese, o processo completo de gravação segue a sequência: (i) abertura do projeto (`.prjx`); (ii) síntese lógica (*Synthesize*); (iii) mapeamento físico (*Place and Route*); (iv) geração do bitstream (*Generate Bitstream*); (v) seleção do programador FlashPro conectado (*Select Programmer*); e (vi) gravação efetiva na FPGA (*Run PROGRAM Action*). Uma vez concluída essa sequência, o hardware descrito nas Seções 5 e 6 passa a operar fisicamente na placa SmartFusion2, pronto para ser validado em conjunto com o firmware do STM32 conforme os requisitos de teste da Seção 4.

8.7 Criação de um Novo Projeto no Libero SoC

Esta subseção apresenta o fluxo completo para a criação de um projeto Libero SoC **do zero**, cobrindo desde a tela de boas-vindas da ferramenta até a etapa de **Synthesize** e a atribuição dos pinos físicos no **I/O Editor**. A partir do momento em que os pinos estiverem corretamente atribuídos e comitados, o fluxo se une exatamente ao tutorial já apresentado na Seção 8: o leitor deve prosseguir diretamente a partir da etapa de **Place and Route** (Seção 8.7.11), repetindo os passos de *Place and Route*, *Generate Bitstream*, seleção do programador *FlashPro* e execução do *Run PROGRAM Action* já descritos anteriormente.

8.7.1 Página de Boas-Vindas

Ao iniciar o Libero SoC v2025.2, a ferramenta apresenta a *Welcome Page*, com atalhos para criar um novo projeto (**New...**) ou abrir um projeto existente (**Open...**), além de projetos recentes e links de documentação (Figura 10).

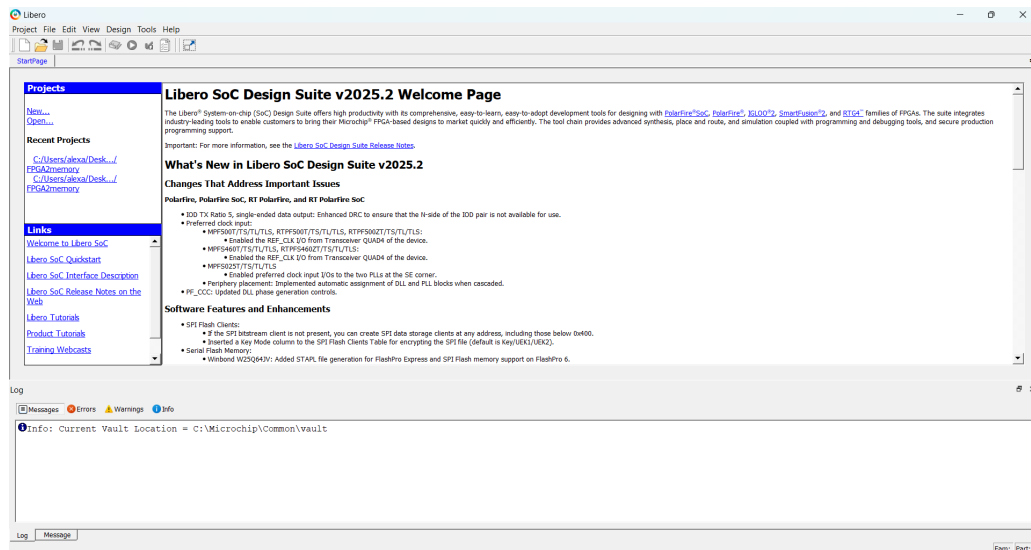


Figura 10: Página de boas-vindas do Libero SoC Design Suite v2025.2.

8.7.2 Detalhes do Projeto (*Project Details*)

Ao clicar em **New...**, o assistente *New project* é aberto na etapa **Project Details**. Aqui são definidos o nome do projeto (**FPGA_design**), o diretório onde os arquivos do projeto serão salvos, e o tipo de HDL preferido (**Verilog**, que também abrange arquivos **.sv** de SystemVerilog). A opção **Enable block creation** deve permanecer desmarcada, pois o projeto não será publicado como um componente reutilizável (Figura 11).

8.7.3 Seleção do Dispositivo (*Device Selection*)

Na etapa seguinte, filtra-se o dispositivo alvo pela família **SmartFusion2**, *die* **M2S025**, encapsulamento **256 VF**, *speed grade* **STD** e tensão de núcleo **1.2 V**. A lista de resultados exibe as duas variantes de temperatura industrial/comercial disponíveis para esse conjunto de filtros; seleciona-se o *part number* **M2S025-VF256I**, correspondente ao dispositivo efetivamente utilizado na placa NASCERR (Figura 12).

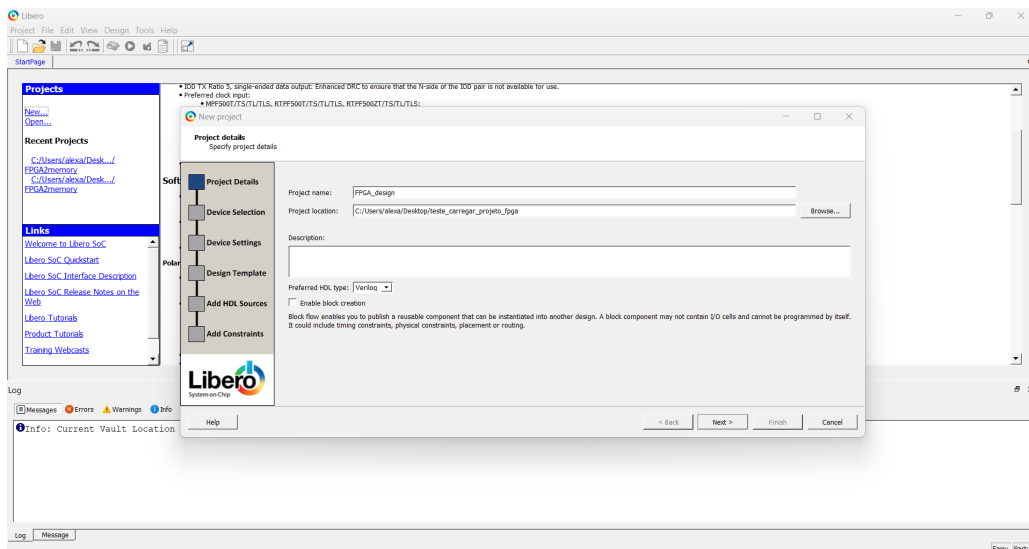


Figura 11: Etapa *Project Details* do assistente *New project*.

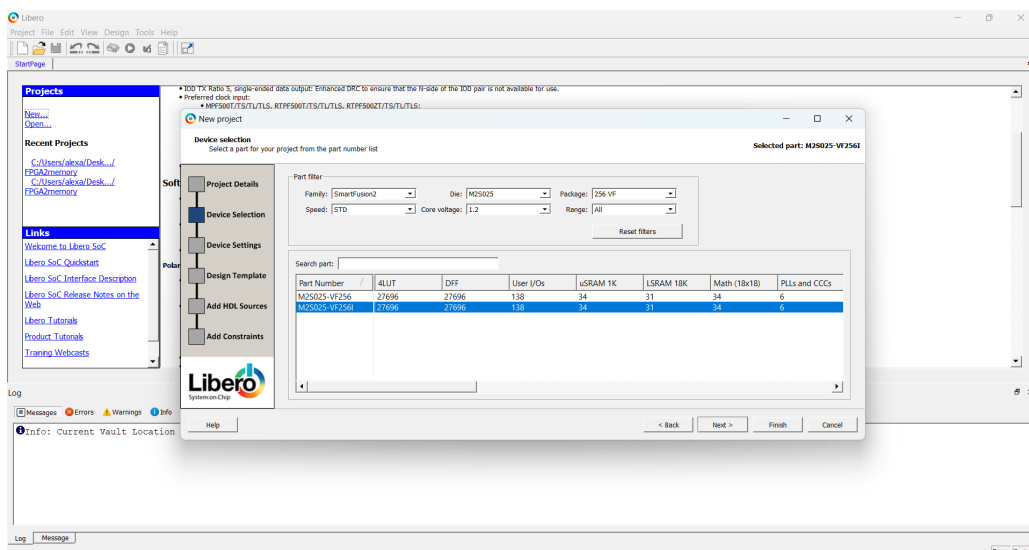


Figura 12: Etapa *Device Selection*, com os filtros de família, *die*, encapsulamento e tensão aplicados.

8.7.4 Configurações do Dispositivo (*Device Settings*)

Nesta etapa, define-se a tecnologia de I/O padrão como **LVC MOS 2.5V** (compatível com a tensão de alimentação dos bancos de I/O utilizados pelos barramentos de dados do projeto) e mantém-se a opção **Reserve pins for probes** habilitada. Nas configurações de alimentação, a tensão de *PLL supply* é mantida em **2.5 V** e o tempo de *ramp* da alimentação em **100ms** (Figura 13).

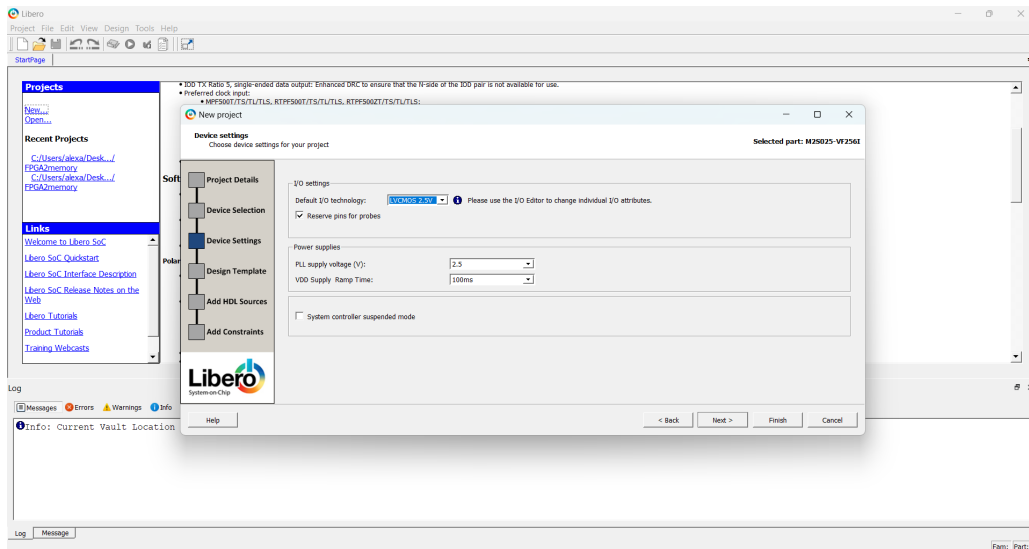


Figura 13: Etapa *Device Settings*: tecnologia de I/O e alimentação.

8.7.5 Modelo de Design (*Design Template*)

Na etapa **Design Template**, seleciona-se a opção **None**. Diferentemente de projetos que utilizam o *System Builder* para instanciar o Microcontroller Subsystem (MSS) da SmartFusion2, o módulo `fpga_top_design` (Seção 5.3) opera inteiramente na *fabric* da FPGA, sem qualquer dependência do Cortex-M interno ou de periféricos do MSS – a comunicação com o processador STM32 externo ocorre diretamente pelos pinos de *fabric* descritos na Seção 6. Por esse motivo, nenhum dos modelos baseados em *System Builder* ou em MSS é necessário (Figura 14).

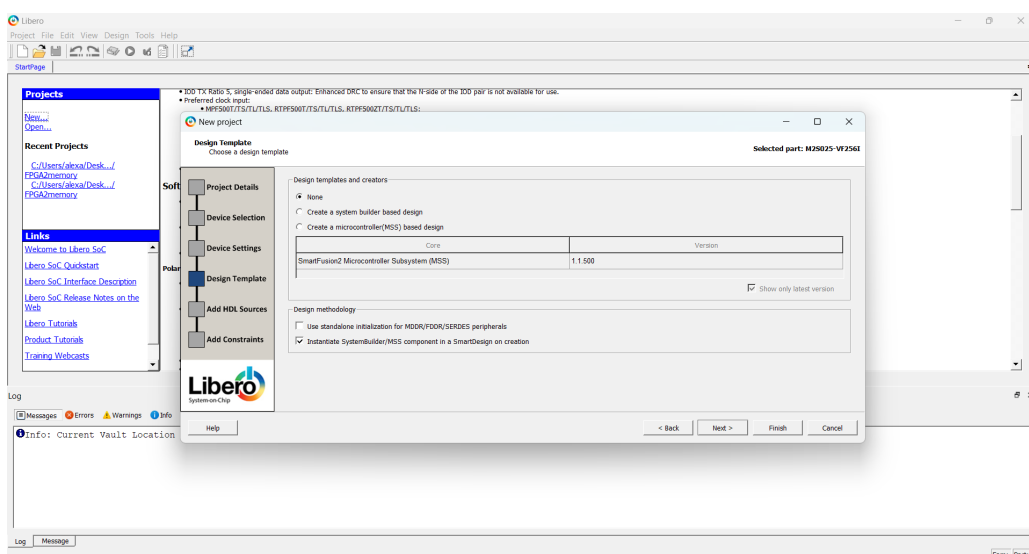


Figura 14: Etapa *Design Template* com a opção **None** selecionada.

8.7.6 Importação dos Arquivos-Fonte (*Add HDL Sources*)

A etapa **Add HDL Sources** é inicialmente exibida vazia (Figura 15). Por meio do botão **Import file**, são selecionados os seis arquivos SystemVerilog que compõem a hierarquia completa do projeto (Seção 5): `design.sv` (Nível 3, top-level), `ecc_design.sv` (Nível 2, multiplexador de ECC), e os quatro codecs de Nível 1: `MRSC_encoder.sv`, `MRSC_decoder.sv`, `TBEC_RSC_encoder.sv` e `TBEC_RSC_decoder.sv` (Figura 16). Após a importação, cada arquivo aparece listado com o status **Imported** e o caminho de origem correspondente (Figura 17).

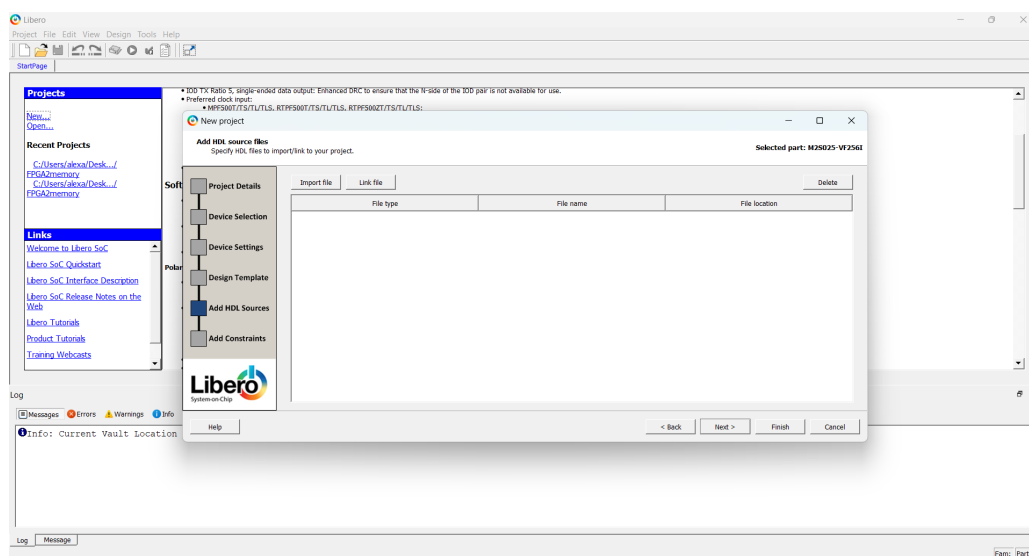


Figura 15: Etapa *Add HDL Sources* antes da importação dos arquivos.

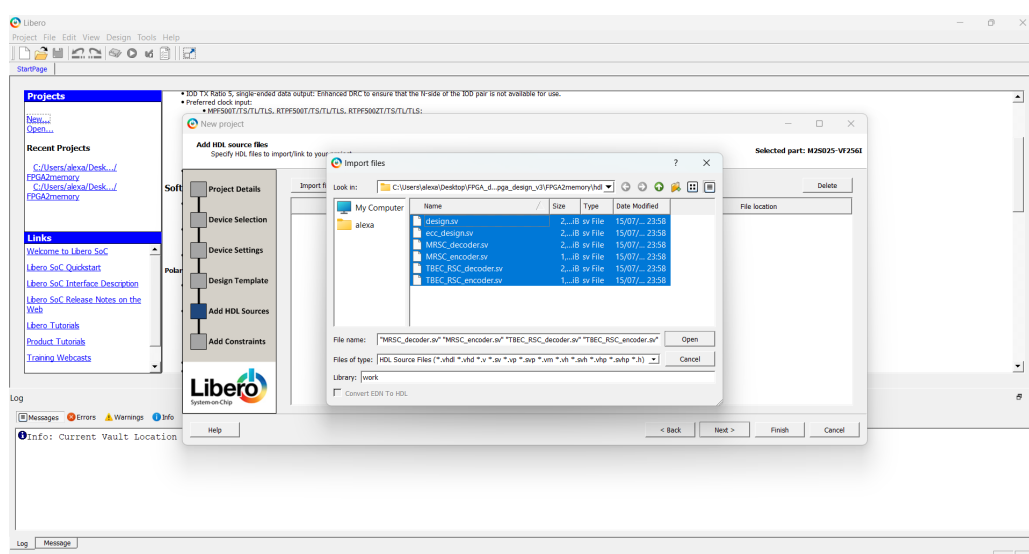


Figura 16: Janela *Import files* com os seis arquivos `.sv` do projeto selecionados.

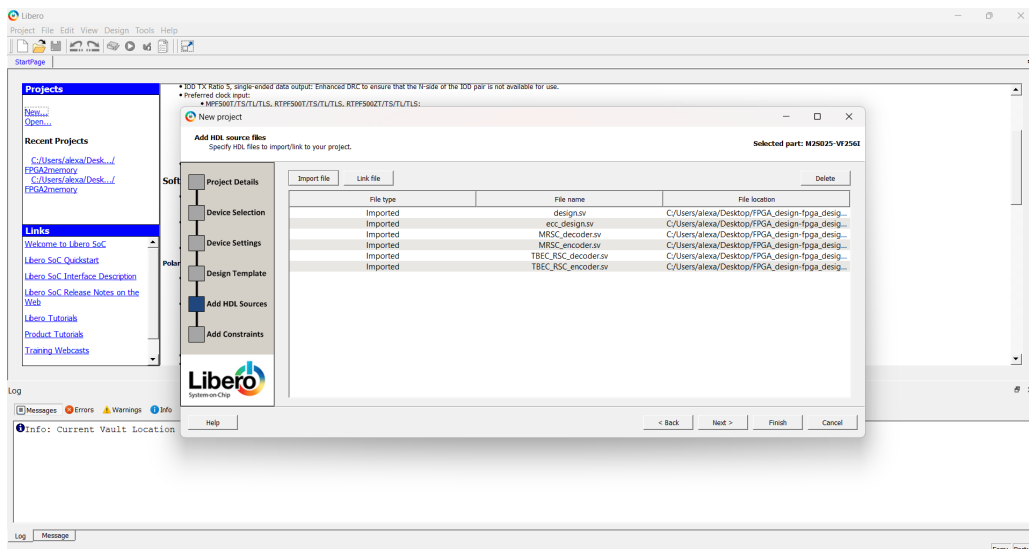


Figura 17: Arquivos-fonte importados com sucesso para o projeto.

Ao clicar em **Finish**, o projeto é criado e a mensagem *"The FPGA_design project was created"* é exibida no painel de **Log** (Figura 18). Nesse momento, o painel **Design Hierarchy** exibe um aviso amarelo *"Please select a root"*, indicando que, embora os arquivos já estejam associados ao projeto, o módulo de topo ainda não foi identificado.

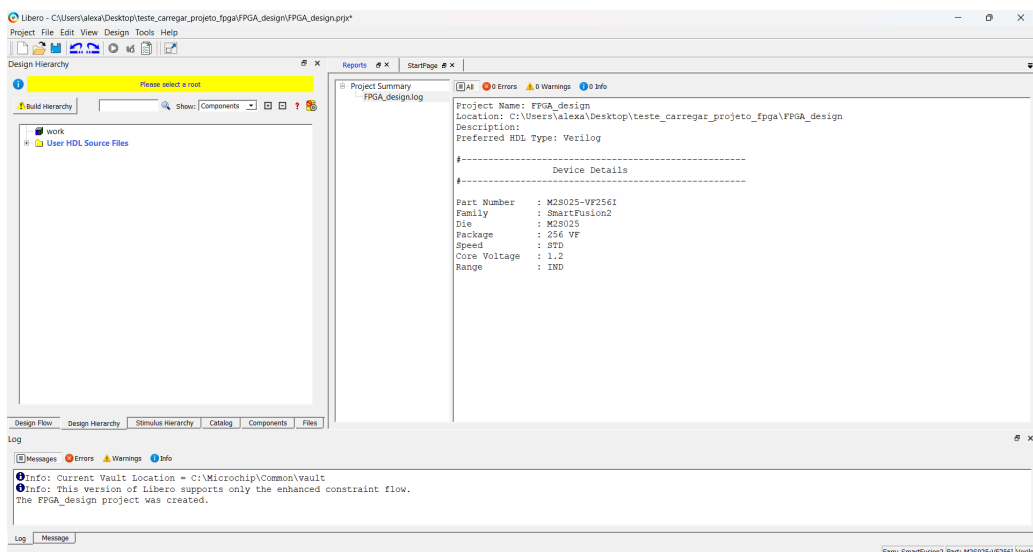


Figura 18: Projeto FPGA_design recém-criado, com o aviso *"Please select a root"*.

8.7.7 Definição do Módulo de Topo (*Set As Root*)

Após acionar o botão **Build Hierarchy**, o Libero interpreta os arquivos-fonte e identifica os módulos SystemVerilog neles declarados, listando o módulo `fpga_top_design`

(definido no arquivo `design.sv`) na raiz da árvore `work`. Clicando com o botão direito sobre esse módulo, seleciona-se a opção **Set As Root** no menu de contexto, definindo-o formalmente como o módulo de topo (Figura 19). O painel de **Log** confirma a leitura de todos os arquivos da hierarquia (`ecc_design.sv`, `MRSC_decoder.sv`, `MRSC_encoder.sv`, `TBEC_RSC_decoder.sv` e `TBEC_RSC_encoder.sv`), sem a ocorrência de erros.

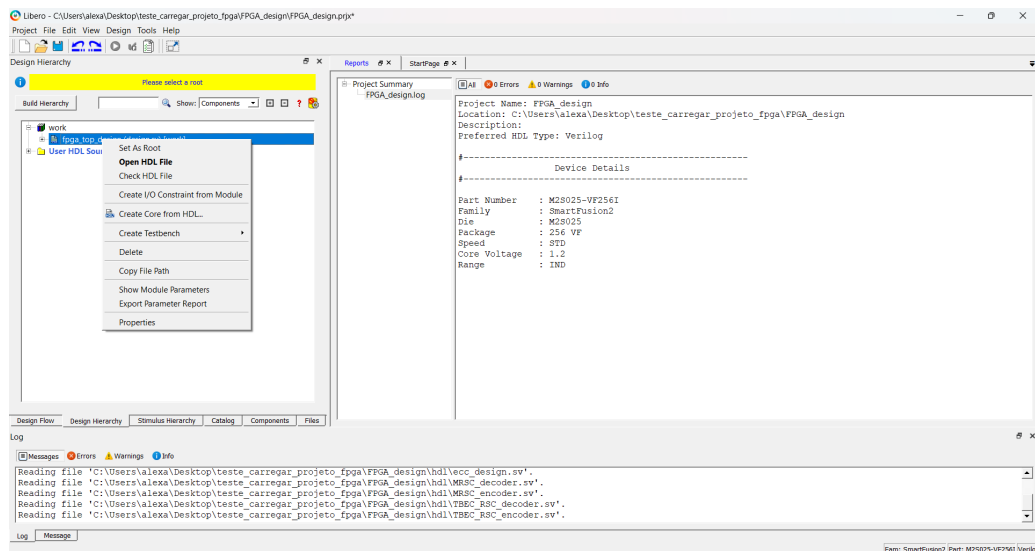


Figura 19: Menu de contexto do módulo `fpga_top_design`, com a opção **Set As Root** destacada.

Uma vez definido o *root*, o aviso amarelo desaparece e o painel **Design Flow** passa a exibir **Top Module(root): fpga_top_design** no topo da árvore de ferramentas, confirmando que o projeto está pronto para prosseguir para a etapa de síntese (Figura 20).

8.7.8 Síntese Inicial e Verificação das Constraints

Com o módulo de topo definido, executa-se a etapa **Synthesize**. O painel de **Log** confirma a conclusão com a mensagem *"Synthesis completed"*. Em seguida, acessa-se o **Constraint Manager**, na aba **I/O Attributes**, que neste ponto do fluxo ainda se encontra vazia – nenhuma constraint de pino foi associada ao projeto até este momento (Figura 21).

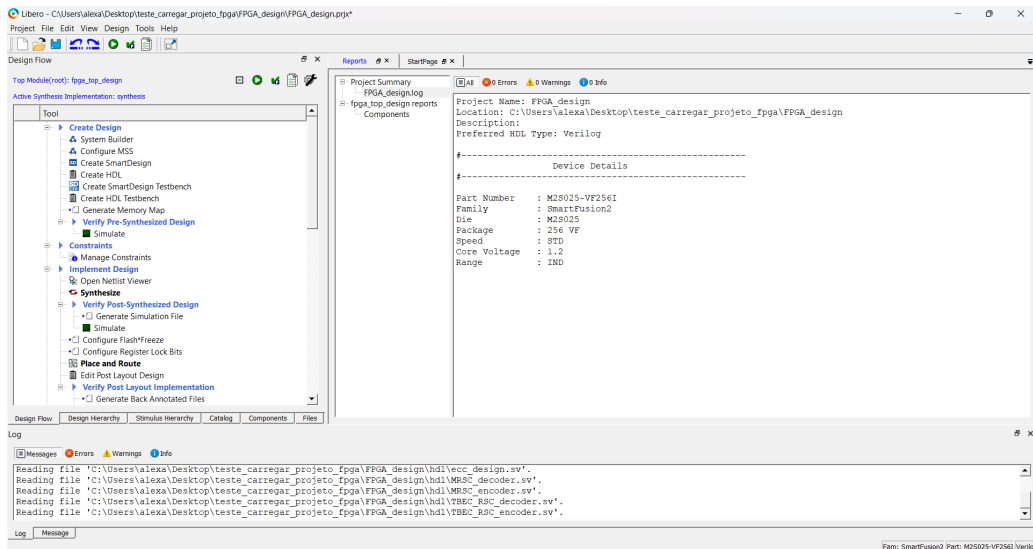


Figura 20: Painel *Design Flow* após a definição do módulo de topo, exibindo *Top Module(root): fpga_top_design*.

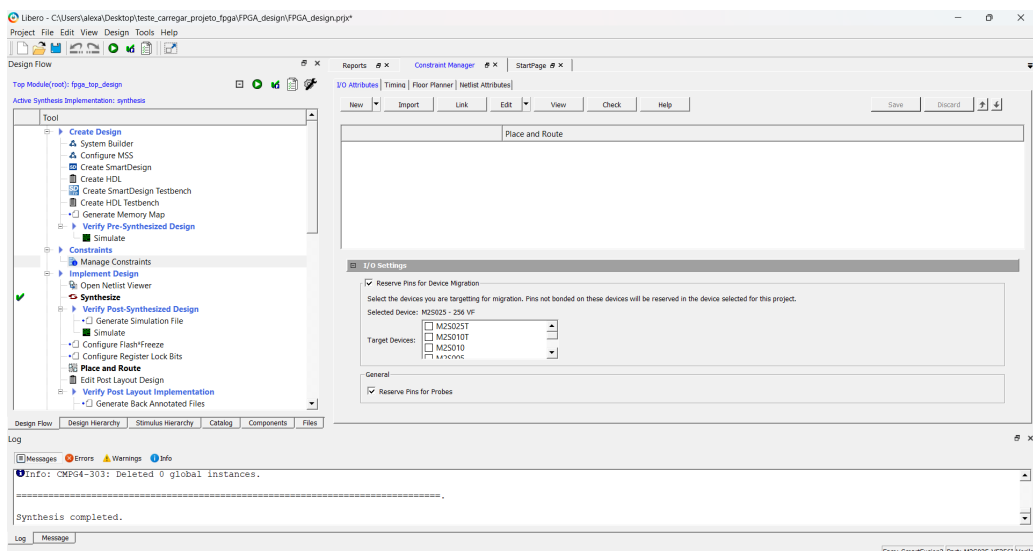


Figura 21: Aba *I/O Attributes* do *Constraint Manager*, ainda sem nenhuma constraint associada, logo após a conclusão da síntese.

8.7.9 Abertura do I/O Editor

Para atribuir os pinos físicos às portas do módulo `fpga_top_design`, clica-se na seta ao lado do botão **Edit**, na barra de ferramentas do **Constraint Manager**, e seleciona-se a opção **Edit with I/O Editor** (Figura 22), abrindo a ferramenta dedicada de edição de pinos.

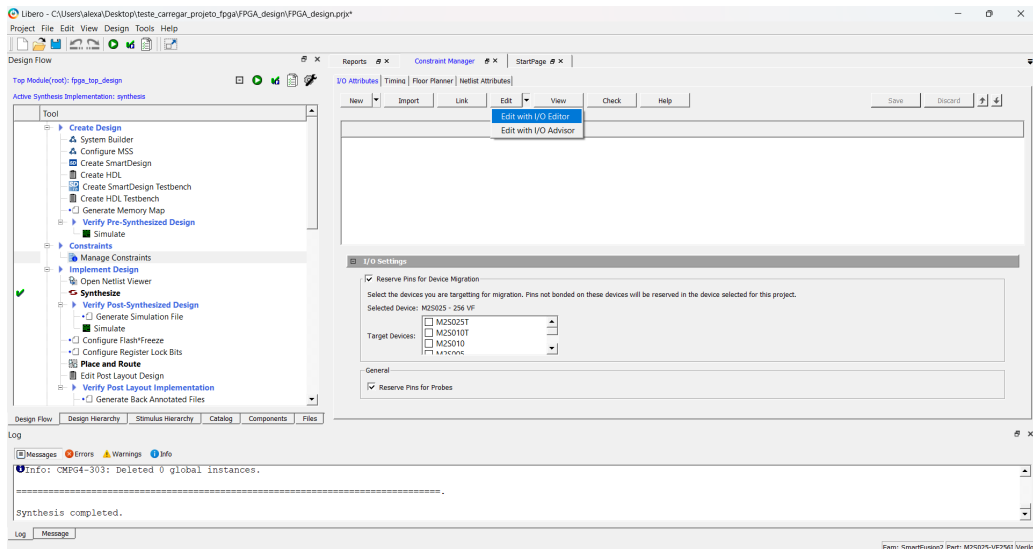


Figura 22: Menu do botão **Edit**, com a opção **Edit with I/O Editor** destacada.

8.7.10 Atribuição dos Pinos no I/O Editor

O **I/O Editor** exibe uma linha para cada porta do módulo `fpga_top_design`, incluindo os barramentos expandidos bit a bit (`ecc_sel[0..2]`, `flag_out[0..2]`, `chip_sel_out[0..1]`, `fpga_mem_io_down[0..15]`, entre outros). Cada pino é atribuído digitando diretamente o número do pacote na coluna **Pin Number** – por exemplo, o pino **G16** é atribuído à porta `ecc_sel[0]` (Figura 23), reproduzindo o mapeamento de pinos apresentado na Seção 6.

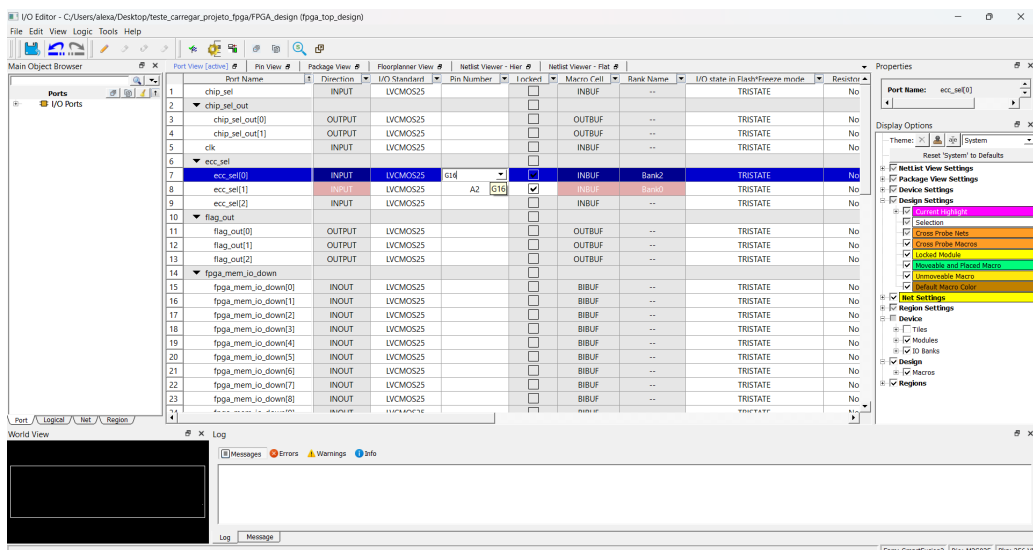


Figura 23: I/O Editor durante a atribuição do pino **G16** à porta `ecc_sel[0]`.

Após a atribuição de todos os pinos e a marcação da coluna **Locked** para fixá-

los, executa-se o comando **Commit**. Nesta etapa, o Libero valida cada valor digitado contra o pacote físico selecionado, rejeitando pinos inexistentes ou incompatíveis – as mensagens "*Pin Number can not be set to illegal value...*" exibidas no painel **Message** (Figura 24) ilustram esse mecanismo de validação, indicando tentativas de atribuição com identificadores de pino inválidos que precisam ser corrigidas antes do *commit* ser aceito para aquelas portas específicas. Ao final, o Libero confirma a checagem das regras de projeto com a mensagem "*Design Rules Check completed successfully*" e grava as novas constraints no arquivo `constraint/io/user.pdc`.

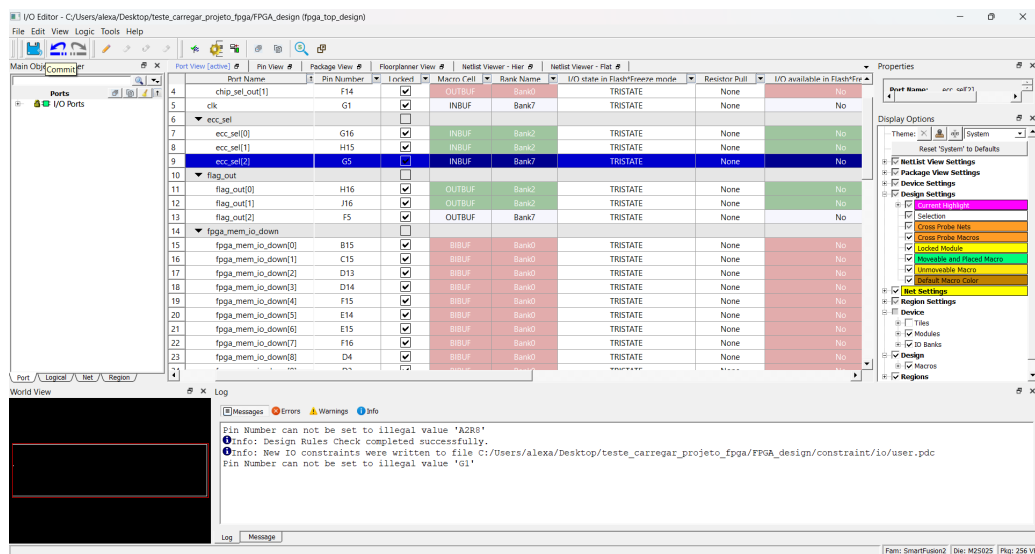


Figura 24: I/O Editor após o *commit* dos pinos, com mensagens de validação exibidas no painel **Message**.

8.7.11 Continuação do Fluxo a partir do Place and Route

Com as constraints de pino corrigidas e comitadas com sucesso – sem nenhuma mensagem de erro remanescente –, o **Constraint Manager** passa a exibir o arquivo `constraint\io\user.pdc` associado como *Target* da etapa **Place and Route** (Figura 25), confirmando que o projeto está pronto para prosseguir.

A partir deste ponto, o restante do fluxo de gravação é idêntico ao já apresentado na Seção 8. O leitor deve prosseguir diretamente a partir da subseção **Place and Route**, executando em sequência: (i) *Place and Route* (Figura 6); (ii) *Generate Bitstream* (Figura 7); (iii) *Select Programmer*, com o FlashPro5 conectado (Figura 8); e (iv) *Run PROGRAM Action*, concluindo a gravação do bitstream na FPGA (Figura 9),

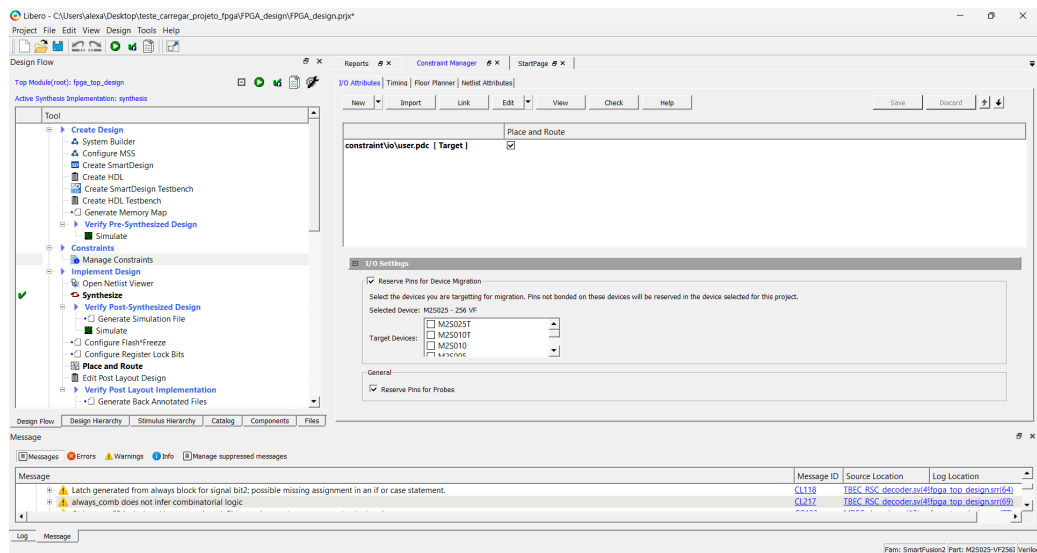


Figura 25: *Constraint Manager* exibindo `user.pdc` associado como alvo (*Target*) da etapa *Place and Route*.

exatamente como descrito nas Seções 8 e seguintes.